



Programa Profesional en Full Stack Developer

Módulo 3. Framework de Front End Angular

Índice

Tema 1. Introducción a Angular

- 1.1. Introducción y objetivos
- 1.2. ¿Cómo funciona Angular?
- 1.3. Características principales de Angular
- 1.4. ¿Cómo trabajamos con Angular?
- 1.5. Herramientas para trabajar con Angular
- 1.6. ¿Cómo instalar Angular?

Tema 2. TypeScript

- 2.1. Introducción y objetivos
- 2.2. Instalación y uso TypeScript
- 2.3. Tipado
- 2.4. Interfaces
- 2.5. Decoradores

Tema 3. Arquitectura de la aplicación

- 3.1. Introducción y objetivos
- 3.2. Partes de una aplicación de Angular
- 3.3. Configuración básica
- 3.4. Arquitectura por componentes
- 3.5. Creación de módulos

Tema 4. Componentes

- 4.1. Introducción y objetivos
- 4.2. ¿Qué es un componente?

4.3. Partes de un componente

4.4. Cómo crear un componente

4.5. Componente principal y componentes hijos. ¿Cómo se enlazan?

4.6. Funciones de ciclo de vida de un componente

Tema 5. Templating

5.1. Introducción y objetivos

5.2. Sintaxis de plantillas

5.3. Hojas de estilos de componente

5.4. Angular signals

Tema 6. Data Binding: One-way and Two-ways

6.1. Introducción y objetivos

6.2. Property Binding

6.3. Two way data binding: FormsModule y [(ngModel)].
Doble comunicación

Tema 7. Event Binding

7.1. Introducción y objetivos

7.2. Creación de eventos en Angular

7.3. \$event

Tema 8. Input y Output

8.1. Introducción y objetivos

8.2. Comunicación entre componentes

8.3. Decorador @Input(). Del padre al hijo

8.4. Decorador @Output(). Del hijo al padre

Tema 9. Control Flow

- 9.1. Introducción y objetivos
- 9.2. Bloque condicional con @if
- 9.3. Bloque de repetición con @for
- 9.4. Bloque condicional @switch

Tema 1. Introducción a Angular

1.1. Introducción y objetivos

A partir de este tema vamos a empezar a descubrir cómo podemos generar aplicaciones web **front-end** a través de una de los frameworks JavaScript más usados del momento.

Angular se ha convertido en uno de los frameworks más populares para el desarrollo de SPA (Single Page Applications).

Existen dos versiones de Angular: **Angular 1** o como se conoce AngularJS y **Angular 2** y sucesivas versiones (actualmente nos encontramos en la 17, el momento de escribir estas líneas). Son mantenidos en gran parte por Google, aparte de una gran comunidad de desarrolladores que aportan sus modificaciones para mejorar el framework.

Nosotros vamos a basar el contenido de este módulo en Angular 2 y versiones posteriores, ya que AngularJS fue una primera versión que no tiene nada que ver con las versiones actuales ni futuras. Su desarrollo se descartó por ser complejo y por apartarse en gran medida del desarrollo por componentes que es hacia donde se mueve el mercado actual. Aquí deberéis tener especial cuidado con buscar información del framework y revisar cuando se esté hablando de Angular 1 (AngularJS) o Angular 2 y sucesivos, ya que podemos encontrar cambios significativos.

A partir de este momento siempre que hablemos de Angular estaremos hablando de la versión actual, que es la 17.

Tema 1. Introducción a Angular

El objetivo de este módulo es adquirir las herramientas necesarias para trabajar con Angular en todas sus versiones desde la 2 en adelante. Para ello vamos a tener que aprender no solo a manejar el framework, sino herramientas como Typescript y patrones de desarrollo como MVC (modelo vista controlador).

Tema 1. Introducción a Angular

1.2. ¿Cómo funciona Angular?

¿Qué es Angular?

Angular es un framework front-end basado en JavaScript y Typescript. Se usa para el desarrollo de aplicaciones web multiplataforma y está pensado para desarrollar aplicaciones del tipo SPA (Single Page Application).

¿Y qué es un SPA? Traducido del inglés sería, una aplicación que ejecuta su contenido en una sola página, pero realmente no es el termino exacto, sería una aplicación web o página web que interactúa con el usuario reescribiendo dinámicamente la página con los nuevos datos que le lleguen del servidor web, esto permite que la carga del contenido sea más rápida y la experiencia de usuario más satisfactoria en cuanto a rendimiento.

Esto viene a sustituir a la forma tradicional de la carga de contenidos donde al cambiar de página se recarga todo el contenido de esta con la perdida de rendimiento que ello produce.

Una SPA funciona cargando el contenido HTML, CSS y Typescript (JavaScript compilado) por completo al abrir la web, al ir pasando de una sección a otra, solo necesita cargar el contenido que cambie nuevo, y no el resto de la página que ya está cargada.

Que sea una única página no implica que tenga todo el contenido cargado en una sección y que no exista distintas vistas. Lo que realmente hace Angular es cargar un módulo que gestiona toda la web y un contenedor principal que se encarga de cargar el contenido dinámico dentro del mismo, haciendo que cambie el aspecto y contenido de la página.

Tema 1. Introducción a Angular

Es un framework que se basa en la creación de aplicaciones a través de componentes. Este concepto es muy práctico ya que nos permite desarrollar aplicaciones complejas a partir de la unión de componentes mucho más simples.

El concepto de desarrollo por componentes es la base para crear aplicaciones web robustas y escalables, así como nos permite un desarrollo más ágil ya que podemos reutilizar los componentes que desarrollamos en varios proyectos.

Tema 1. Introducción a Angular

1.3. Características principales de Angular

Las principales características en las cuales se basan todos los desarrollos que realicemos con Angular son las siguientes:

- ▶ Es un framework que me permite desarrollar **aplicaciones escalables y robustas**, ya que al estar bien compartimentado sabes donde esta cada cosa y separa muy bien la lógica, de los estilos y del interfaz.
- ▶ El desarrollo por componentes y la organización robusta que nos propone Angular me permite la **separación de tareas en grupos de trabajo**, con lo que asume una agilidad mucho mayor en el desarrollo y una reducción en los costes de producción.
- ▶ Es bastante **fácil de mantener y actualizar**.
- ▶ Al basarse en desarrollo por componentes el ciclo de vida de las aplicaciones es más largo ya que podemos reutilizar nuestros desarrollos en otros proyectos con muy pocos cambios o casi ninguno.
- ▶ El rendimiento y la **carga de la aplicación es muy rápida**.
- ▶ Angular sirve para proyectos sencillos de una web con un único desarrollador hasta una aplicación web de ámbito empresarial donde varios programadores tengan asignados diferentes roles y tareas.

Tema 1. Introducción a Angular

1.4. ¿Cómo trabajamos con Angular?

En el siguiente enlace se puede encontrar esta y otra información relevante sobre Angular, todo lo explicado en estas líneas está fundamentado en la documentación oficial del framework.

<https://angular.dev/>

Vamos a argumentar en unos pocos puntos cada una de las ventajas que me aporta trabajar con un framework como Angular, que van muy ligadas de la mano con las características que hemos comentado en el punto anterior.

Generación de código

Angular transforma las plantillas en código altamente optimizado para el trabajo en una máquina virtual JavaScript. Nos permite tener todos los beneficios de la creación manual de código, pero con la potencia de cualquiera de los mejores frameworks.

Universal

Es multiplataforma y dispositivo, el código generado es puramente HTML y CSS, por lo que podemos renderizarlo en todos los navegadores.

División de código

El usuario únicamente carga la parte de la página que necesita usar en cada momento, con lo que ahorramos muchos tiempos de carga. La creación de componentes reutilizables favorece la velocidad de carga en el navegador.

Tema 1. Introducción a Angular

Sistema de plantillas

Angular posee un sistema de plantillas que me permite separar totalmente el interfaz visual (HTML y CSS), de la lógica (Typescript) de una forma muy simple y sin tener que capturar los elementos del DOM como en JavaScript.

Angular CLI

Es el cliente de angular que se ejecuta por terminal de línea de comandos y que nos aligera tanto la creación de proyectos de Angular como la de los componentes, así como levantar el servidor de trabajo o construir los archivos que posteriormente subiremos a al servidor para publicar nuestra aplicación en internet.

Testing

Angular integra las librerías más avanzadas del mercado para la creación de test unitarios, lo que favorece la creación y ejecución de estas pruebas sobre nuestro código.

Tema 1. Introducción a Angular

1.5. Herramientas para trabajar con Angular

Antes de instalar Angular y empezar a trabajar con él, necesitamos tener una serie de herramientas que son absolutamente necesarias y que sin ellas no podríamos desarrollar nuestro trabajo.

Muchas de ellas ya la venís usando en los temas anteriores, pero no viene mal recordar algunas y otras nuevas con las que necesitaremos trabajar.

Lo primero que necesitamos es un editor de código o IDE para ello vamos a usar Visual Studio Code, aquí os dejo el enlace de descarga. Es un editor de código gratuito que tiene un sistema de plugins muy completo que aumenta sus posibilidades del editor hasta el infinito.

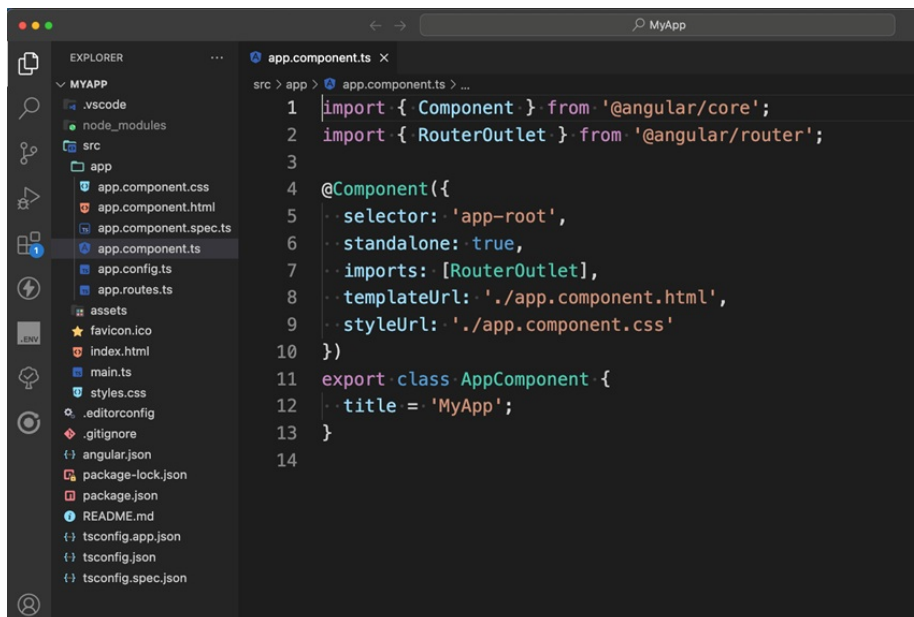


Figura 1. Árbol de ficheros de Angular. Fuente: elaboración propia.

Dentro de las extensiones de Visual Studio Code os recomiendo que instaléis Angular Language Service ya que es un plugin que me ayuda mucho en la escritura de código con Angular.

Tema 1. Introducción a Angular

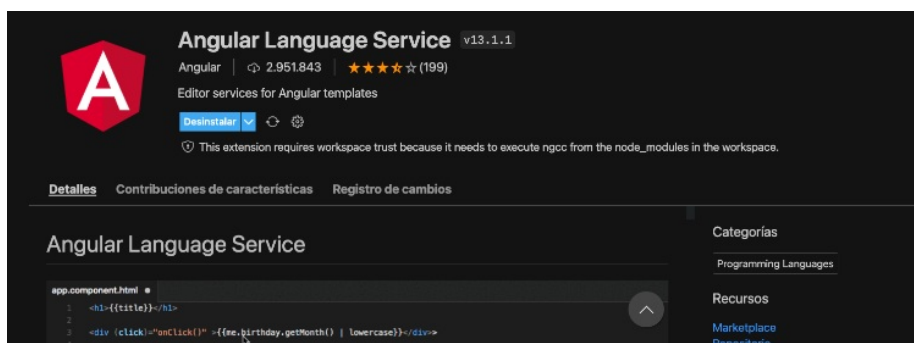


Figura 2. Plugin de Visual Studio Code. Fuente: elaboración propia.

Otra herramienta indispensable para nosotros es Node que, aunque tendrá un tema a parte dentro de este Máster lo necesitamos porque con Node se instala un gestor de paquete llamado NPM que es vital para poder instalar el cliente de Angular que me permitirá crear aplicaciones con este framework.

Para instalar Node nos iremos a su página oficial y nos descargaremos el instalable para nuestro sistema operativo a través del siguiente enlace:

<https://nodejs.org/en/download/>

Tema 1. Introducción a Angular

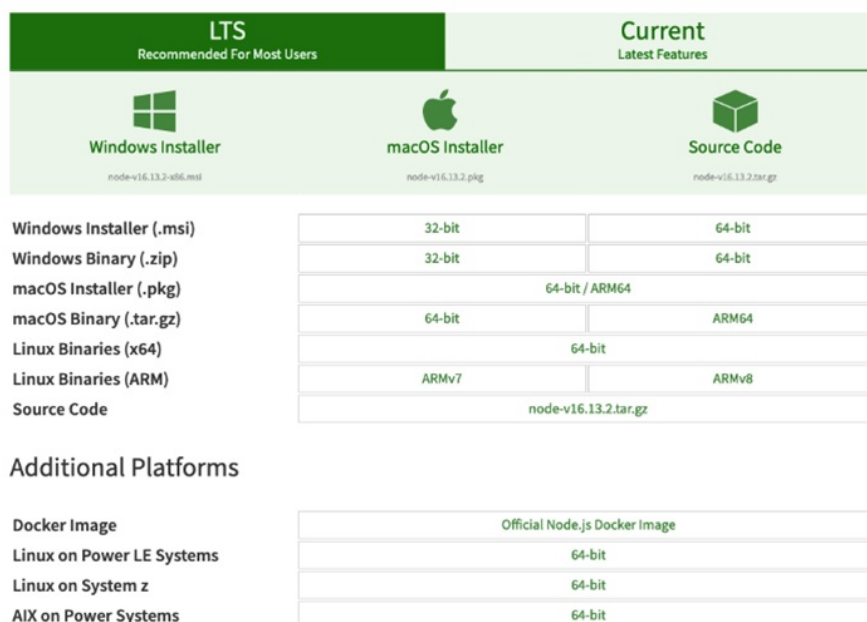


Figura 3. Captura página web de Node JS. Fuente: elaboración propia.

De todas las opciones para nuestro sistema operativo y la versión LTS seleccionamos la que mejor nos convenga. Se descargará y ejecutará un instalador estándar el cual nos pedirá una serie de confirmaciones para proceder a la instalación.

Para comprobar que todo ha terminado de manera correcta podemos abrir una instancia del terminal dentro de nuestro ordenador y lanzar el comando.

```
node --version
v14.17.1
```

Si el resultado de esta ejecución nos devuelve información sobre la versión que hemos instalado previamente quiere decir que hemos completado la instalación de NodeJS correctamente en nuestro ordenador.

A partir de este momento y podemos usar el gestor de dependencias NPM para instalar cualquier librería de JavaScript necesaria para trabajar, incluido Angular.

Tema 1. Introducción a Angular

1.6. ¿Cómo instalar Angular?

Para trabajar con Angular vamos a tener que instalar cliente de Angular o lo que se conoce como Angular CLI, es una herramienta de línea de comandos que me simplifica mucho las acciones a la hora de generar un nuevo proyecto o crear nuevos componentes, será una herramienta que usaremos continuamente así que debemos perder el miedo a usarla desde el principio.

Angular CLI realiza una serie de tareas para que no tengas problemas. Aquí hay unos ejemplos:

ng build	Compila una aplicación Angular en un directorio de salida llamado dist/ en la ruta de salida dada. Debe ejecutarse desde dentro de un directorio de espacio de trabajo.
ng serve	Me permite levantar el servidor de Desarrollo para arrancar la aplicación y visualizar los cambios que voy desarrollando en tiempo real.
ng generate	Me permite generar componentes, servicios, pipes y cualquier estructura necesaria para trabajar en angular. Gracias a este script podemos crear y vincular los componentes con sus archivos de vistas y html con mucha facilidad.
ng test	Nos permite generar pruebas unitarias dentro de un proyecto determinado.

Tabla 1. Tareas de Angular CLI. Fuente: elaboración propia.

Tema 1. Introducción a Angular

Para instalar Angular Cli solo tienes que usar el gestor de paquetes NPM que hemos instalado anteriormente desde la terminal o línea de comandos. Abre el CMD o terminal y ejecuta la siguiente instrucción.

```
npm install -g @angular/cli
```

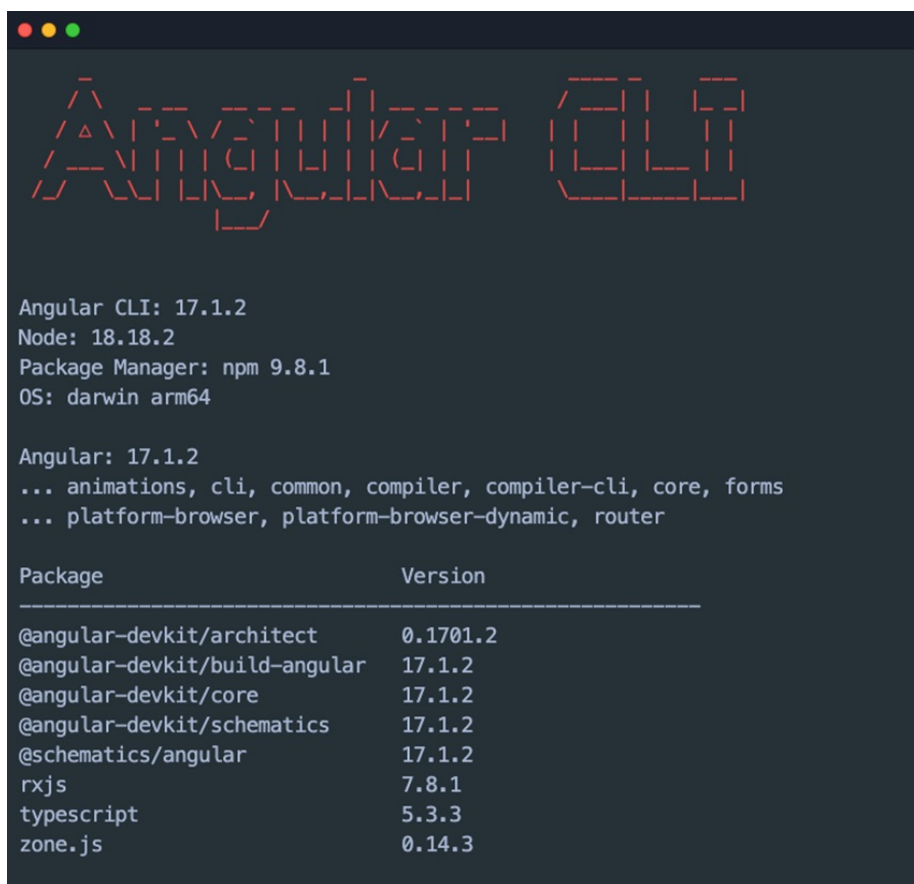
Este comando lo que me permite instalar el Angular CLI de forma global, de ahí el -g, dentro de nuestro equipo.

Para comprobar que angular este correctamente instalado en nuestro equipo escribimos el siguiente comando.

```
ng versión
```

Que deberá lanzarnos como respuesta algo parecido a lo que vais a ver en la imagen siguiente.

Tema 1. Introducción a Angular



```
Angular CLI: 17.1.2
Node: 18.18.2
Package Manager: npm 9.8.1
OS: darwin arm64

Angular: 17.1.2
... animations, cli, common, compiler, compiler-cli, core, forms
... platform-browser, platform-browser-dynamic, router

Package                                  Version
-----
@angular-devkit/architect                0.1701.2
@angular-devkit/build-angular            17.1.2
@angular-devkit/core                     17.1.2
@angular-devkit/schematics               17.1.2
@schematics/angular                     17.1.2
rxjs                                     7.8.1
typescript                               5.3.3
zone.js                                  0.14.3
```

Figura 4. Captura terminal ng -version. Fuente: elaboración propia.

Una vez instalada esta herramienta y las anteriores ya podemos empezar a crear nuestras aplicaciones en Angular.

Crea una carpeta dentro de un directorio de trabajo a tu elección, y dentro de esa carpeta, desde la terminal o línea de comandos, ejecuta al comando que te permitirá crear una aplicación de Angular.

Dicho comando es:

```
ng new primeraApp
```

Donde **primeraApp** corresponde al nombre que le pongamos a nuestra aplicación de angular.

Tema 1. Introducción a Angular

Este comando ejecutará un conjunto de órdenes que te realizará una serie de preguntas. La primera pregunta será qué motor de hoja de estilos queremos usar. Moviéndonos con las flechas, seleccionaremos en primera instancia CSS (igualmente más adelante realizaremos desarrollos con SCSS que habéis visto en capítulos anteriores).

```
Cursos/06_UNIR/Actualización_2024
→ ng new primeraApp
? Which stylesheet format would you like to use? (Use arrow keys)
> CSS
  SCSS  [ https://sass-lang.com/documentation/syntax#scss ]
  Sass  [ https://sass-lang.com/documentation/syntax#the-indented-syntax ]
  Less  [ http://lesscss.org ]
```

Figura 5. Captura terminal creación de proyecto de Angular. Fuente: elaboración propia.

Además de la pregunta anterior, antes de la instalación de la aplicación, el terminal nos pregunta si queremos activar la **renderización del lado del servidor (SSR)**. De momento respondemos que no, para poder centrarnos en los conceptos más simples, y dejaremos para más adelante las configuraciones complejas que nos ofrece Angular.

Una vez acabe el proceso de carga de la aplicación en Angular (puede tardar tiempo dependiendo del equipo con el que trabajes y tu conexión a Internet).

Lo que está ocurriendo en este proceso es que se está creando una carpeta llamada primeraApp, una vez termine este proceso nos meteremos en la carpeta que Angular ha creado y allí ejecutaremos el comando:

```
cd primeraApp
ng serve
```

Este comando lo que hará será compilar el código Typescript y levantar un servidor en local que me permitirá ver la aplicación funcionando.

Tema 1. Introducción a Angular

La aplicación por defecto se levanta en un puerto del ordenador que es un puerto local 4200. Así que si vas a un navegador y pones:

<http://localhost:4200>

Veras que una aplicación se levanta que tiene más o menos este aspecto.

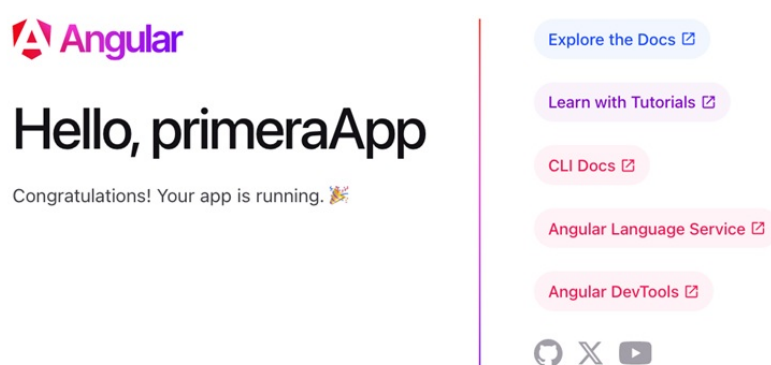


Figura 6. Pantalla inicial de un proyecto de Angular. Fuente: elaboración propia.

Esta es la aplicación por defecto que arranca angular la primera vez que levantamos el servidor.

Dentro de la carpeta de trabajo vamos a ver como Angular crea un sistema de carpetas similar al de la imagen, en este sistema puedes encontrar todos los archivos necesarios para que la aplicación de angular funcione.

Tema 1. Introducción a Angular

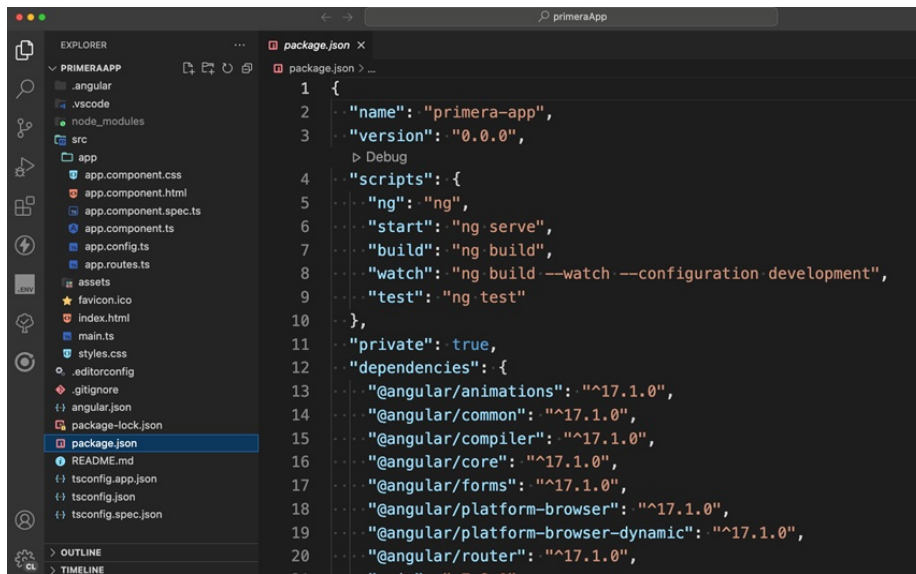


Figura 7. Árbol de ficheros de Angular, fichero package.json. Fuente: elaboración propia.

Vamos a hacer una introducción rápida de algunos ficheros importantes para saber para qué sirven, aunque, según avance el curso, iremos ampliando la mayoría de ellos.

Directorio node_modules

Contiene todas las dependencias del proyecto. Este directorio no tenemos que editarlo nosotros ya que NODE lo hará por nosotros.

En caso de que necesitemos agregar una dependencia o librería a nuestro proyecto de angular lo haremos por medio del comando.

```
npm install <paquete> --save
```

La bandera `--save` es opcional, pero se usa para marcar si la dependencia que deseamos instalar es necesaria solo para el desarrollo de la aplicación, es decir si ponemos el `--save` la librería instalada es necesaria solo para desarrollo con lo que cuando publiquemos en producción no será necesaria tenerla. La información de todos los paquetes del proyecto se pueden ver en el archivo `package.json`.

Tema 1. Introducción a Angular

Directorio src

Contiene el código de nuestra aplicación, en este directorio es donde nosotros escribimos nuestro código. Los archivos más importantes son el archivo `style.css`, y el `index.html` que es el archivo donde corre toda nuestra aplicación.

Directorio src/app

Contiene los archivos que son responsables del funcionamiento de nuestra aplicación. Dentro de esta carpeta cabe resaltar el archivo `app.config.ts`, dentro del cual encontramos la configuración general de la aplicación, así como la asignación del array en el que se encuentran las rutas a partir de las cuales generaremos nuestro flujo de trabajo.

Directorio src/assets

Contiene los archivos estáticos de la aplicación como imágenes, archivos audio etc.

Otro archivo es `angular.json`, este es esencial, ya que tiene muchas propiedades. Va a permitir gestionar el espacio de trabajo, así como las configuraciones de los diferentes proyectos de la aplicación Angular. Es importante conocer las configuraciones principales con el objetivo de asegurarse del correcto dominio de un proyecto realizado con ayuda de Angular CLI.

Por ejemplo, en este archivo podemos cambiar donde se cargan los archivos css y donde están alojadas las librerías externas de terceros en javascript.

Tema 1. Introducción a Angular

La modificación de este fichero requiere que volvamos a ejecutar el servidor a través de:

```
ng serve
```

Recuerda que si el servidor esta levantado, te dejará la terminal bloqueada, no te preocupes por eso es correcto, cada vez que realices algún desarrollo dentro de la app la terminal volverá a recompilar los archivos de trabajo y recargará la aplicación en el navegador de forma automática.

Para parar el servidor y volver a recuperar el uso de la terminal deberás pulsar estas teclas:

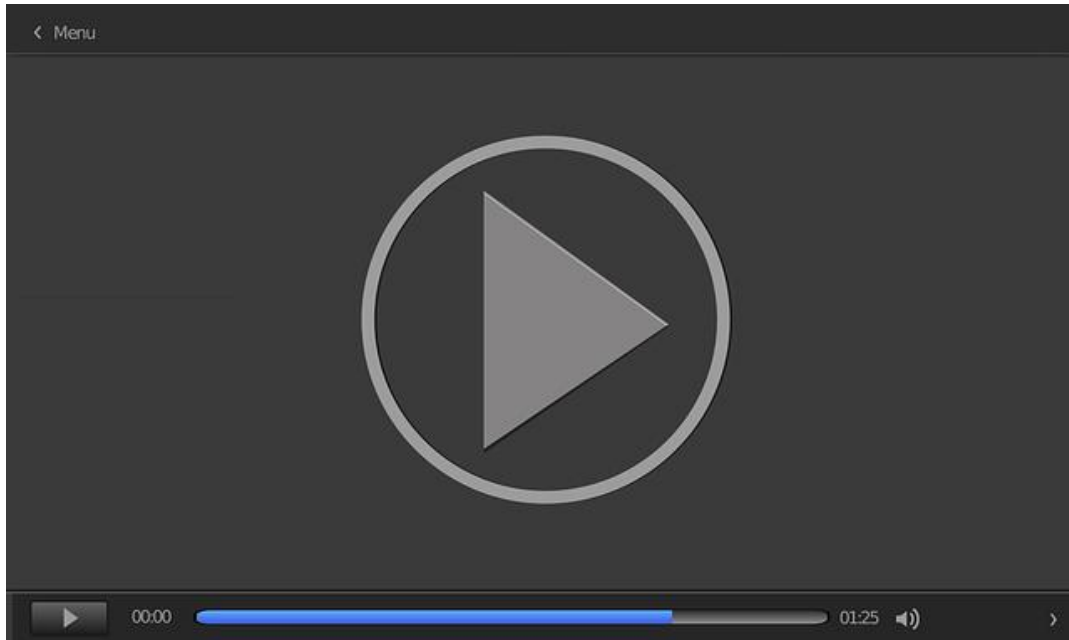


Figura 8. Combinación de teclas para parar un proyecto de Angular. Fuente: <https://necesitoayuda.eu/>

Y hasta aquí el primer acercamiento a una aplicación en Angular, durante el desarrollo de este módulo iremos desengranando poco a poco estos y más puntos que nos permitirán desarrollar aplicaciones web cada vez más complejas y funcionales.

Tema 1. Introducción a Angular

En el siguiente vídeo, *Instalación de Angular y creación de un nuevo proyecto*, se explica cómo instalar Angular y cómo se crea un proyecto desde terminal.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=e2eb6588-8dfa-440a-881b-b1270149baff>

2.1. Introducción y objetivos

JavaScript, como lenguaje, se enfrenta a algunas limitaciones importantes que se han ido solucionando con las nuevas versiones del estándar web.

Algunas de ellas como el tipado débil de las variables, el uso de estructuras de datos más complejas o el aviso de errores en tiempo de desarrollo no de ejecución son algunos puntos que JavaScript hoy en día no ha conseguido solventar y que TypeScript nos ayuda solventarlo

El hecho de que estudiemos TypeScript en este curso es porque angular lo usa como piedra fundamental del desarrollo de sus componentes, así que este tema intenta servir como punto de partida y toma de contacto para aprender a trabajar con TypeScript.

Con TypeScript nos enfrentamos a un conjunto de herramientas creadas por encima de JavaScript.

Un resumen muy escueto de cómo funciona TypeScript, es un lenguaje compilado, este tipo de lenguajes necesitan de una herramienta para transformar código TypeScript en JavaScript, y en Angular esta herramienta ya viene implementada y si lanza desde:

```
ng serve
```

El objetivo fundamental de este tema es aprender los conceptos más importantes de TypeScript y como se aplica para usarlo como herramienta de desarrollo en nuestras aplicaciones de angular.

2.2. Instalación y uso TypeScript

Aunque TypeScript en Angular no necesita de ninguna instalación es cierto que TypeScript se puede usar no solo en Angular si no también en solitario para el desarrollo con JavaScript o en otros frameworks como React, Nest JS, etc.

Aprender TypeScript no es muy difícil si ya sabes JavaScript y te aporta herramientas muy interesantes.

La manera más fácil de usar TypeScript de forma nativa es instalarlo por defecto dentro de nuestra máquina de trabajo

```
npm install -g typescript
```

Recuerda que la bandera -g significa que no instalamos TypeScript dentro de una carpeta concreta si no de forma global en nuestro equipo.

Para estar seguros de que TypeScript se ha instalado correctamente tenemos que aplicar el siguiente comando en nuestra terminal. La respuesta esperada es la versión de TypeScript actual que acabamos de instalar.

```
tsc --version
```

Puntos a tener en cuenta cuando se usa TypeScript

Los ficheros con los que vamos a trabajar tendrán extensión .ts no extensión .js como estábamos usando hasta ahora.

Para poder trabajar con ellos debemos compilarlos y transformarlos a JavaScript con el siguiente comando.

```
tsc nombre_fichero.ts
```

2.3. Tipado

Una de las mejoras que trae TypeScript respecto a JavaScript es el tipado fuerte de variables. Este es opcional pero muy recomendable por parte de los desarrolladores ya que nos ayuda a cometer menos errores a la hora de programar.

Por qué es tan importante el tipado para los desarrolladores:

- ▶ Mediante el uso de los diferentes tipos a la hora de definir nuestras variables, limitamos su uso.
- ▶ Las variables que creemos solo vamos a poder utilizarlas para almacenar valores de esos tipos en concreto.
- ▶ Para especificar el tipo de unas variables podemos hacerlo con la siguiente estructura.

```
let nombreVariable: tipo = valor  
let nombre: string = "Juan"
```

Diferentes tipos de datos en TypeScript y cómo se usan

Boolean

Se trata del tipo más básico, en el que especificamos si el valor es verdadero (true) o falso (false).

```
let termina: boolean = false
```

Number

Es la definición genérica que usamos para hacer referencia a cualquier tipo de número en TypeScript.

Aparte de eso, el tipo `number` también soporta el uso de binarios, octales y hexadecimales.

```
let decimal: number = 23
let decimal2: number = 12.3
let hex: number = 0xf00d
let binario: number = 0b1001
let octal: number = 0o744
```

String

Utilizamos el tipo `string` para hacer referencia a todas las cadenas de tipo literal.

Podemos limitar estas cadenas con comillas dobles ("), simples (') o podemos utilizar los template strings típicos de JavaScript (`).

```
let nombre: string = "Roberto"
let apellidos: string = 'García'
let saludo: string = `Hola, me llamo ${nombre} ${apellidos}`
```

Array

Del mismo modo que pasa en JavaScript, los arrays nos permiten almacenar varios valores dentro de una estructura.

La diferencia aquí es que vamos a especificar el tipo de los valores almacenados en nuestro `array`.

Se puede especificar de dos maneras:

```
let list: number[] = [1,2,3,4,5]
let list2: Array<number> = [6,7,8,9]
```

Tupla

Este tipo de dato nos permite representar un array que, normalmente dispone un tamaño fijo y en el que limitamos a dos únicos tipos los datos que vamos a poder incluir dentro del mismo. Lo vemos con un ejemplo:

```
let t: [string, number]
t = ['hola', 32]
// Únicamente recibe string y number
```

Se podrían incluir más elementos de los tipos especificados, pero se suele limitar su uso a pares de valores.

Enum

Utilizamos este tipo de dato específico para aplicar un nombre más amigable a un conjunto de datos numéricos.

```
enum Size {Small, Medium, Big}
let s: Size = Size.Medium
```

Por defecto, los elementos de tipo enum comienzan a definir sus valores a partir de 0, pero podemos cambiar ese comportamiento, modificando alguno de los valores. Vemos cómo modificar el valor de inicio por defecto.

```
enum Size {Small=1, Medium, Big}
let s: Size = Size.Medium
console.log(s) // 2
```

Incluso podríamos definir todos los valores.

```
enum Size {Small=3, Medium=6, Big=9}
let s: Size = Size.Medium
console.log(s) // 6
```

Podemos recuperar también el nombre relacionado con alguno de los valores:

```
enum Size {Small=3, Medium=6, Big=9}
let nbColor: string = Size[6]
console.log(nbColor) // Medium
```

Any

Utilizamos este tipo para definir aquellas variables de las cuales desconocemos su tipo.

Este tipo de variables suelen venir del análisis de contenido dinámico o de librerías de terceros que desconocemos.

```
let duda: any = 4
duda = "almacenamos un string"
duda = true

console.log(duda) // true
```

Funciones

En typescript las funciones que reciben o que devuelven parámetros también pueden tipar los datos tanto en un sitio como en otro.

```
function sumar(pNumeroA: number, pNumeroB: number): number {
    let resultado: number = pNumeroA + pNumeroB;
    return resultado;
}
```

Cuando una función no devuelve nada usamos el tipo de dato contrario a **ANY** que es **VOID**.

```
function saludo(): void{
    console.log("Hola Mundo!!")
}
```

Muchas veces necesitamos trabajar de manera concreta con variables de las cuales desconocemos su tipo.

Para poder decirle al compilador que «confíe en nosotros» y trate a las variables con el tipo que necesitamos, podemos hacerlo de dos maneras diferentes.

```
let valor: any = "Es una cadena"
```

```
let sizeValor: number = (<string>valor).length
```

```
let sizeValor2: number = (valor as string).length
```

2.4. Interfaces

Técnicamente un interfaz es un mecanismo de la programación orientada a objetos que trata de suplir la carencia de la herencia múltiple. Casi todos los lenguajes de programación que implementan la programación orientada a objetos no ofrecen la posibilidad de definir una clase que extienda de varias clases a la vez y muchas veces sería lo deseable. Y es aquí donde los interfaces cobran su uso.

Mediante las interfaces de TypeScript podemos definir contratos que, las clases que implementen dichos interfaces tendrán que cumplir.

De esta manera nos aseguramos de que las clases que implementan ciertos interfaces van a tener los mismos métodos y propiedades, aparte de las suyas propias.

```
interface IPersona{
    name: string
    getName(): string
}

class Persona implements IPersona{
    name: string
    getName(): string{
        return this.name
    }
}
```

Si en el ejemplo anterior, la clase Persona no cumple con alguno de los nombres o tipos de las propiedades o métodos definidos en el interfaz, obtendremos un error de Typescript indicándonos que algo va mal en nuestro desarrollo.

Tema 2. TypeScript

Como has podido ver los interfaces no son difíciles de definir y usar. En el mundo de la programación orientada a objetos se usan bastante y para nosotros suponen una ayuda a la hora de definir que métodos o que propiedades va a tener nuestra clase.

El uso de interfaces nos obliga a mantener buenas prácticas de programación y a cambio nos provee de una serie de mecanismos que nos permite detectar errores de una forma más fácil.

2.5. Decoradores

Un decorador es una función que, dependiendo de que cosa queramos decorar, sus argumentos serán diferentes.

Usan la forma `@expression` donde «expression» evaluará la función que será llamada. A continuación, explicaré los decoradores más frecuentes:

Decoradores de clase

Es aplicado al constructor de la clase y puede ser usado para observar, modificar o reemplazar la definición inicial de la clase. Su único argumento es `target` que vendría siendo la clase decorada, tipado como `Function` o `any`:

```
function Bienvenida(target: Function): void{
    target.prototype.saludo = function(): void{
        console.log("Hola Mundo")
    }
}
```

```
@Bienvenida
class Saludar{
    constructor(){}
}
```

```
let s = new Saludar()
s.saludo()
```

Si quisiéramos modificarlo, agregando un parámetro en la llamada al decorador, podríamos hacerlo de la siguiente manera

```
function Bienvenida(saludo: string){
    return function(target: Function){
        target.prototype.saludo = function(): void{
            console.log(saludo)
        }
    }
}
```

```
@Bienvenida("Saludando")
```

```
class Saludar{  
    constructor(){}  
}
```

```
let s = new Saludar()  
s.saludo()
```

Podemos decorar de la misma manera, propiedades definidas en nuestras clases.

Primero definimos la función decoradora.

```
function LogCambios (target: Object, key: string){  
    var valueProp: string = this[key]  
    if(delete this[key]){  
        Object.defineProperty(target, key, {  
            get: function(){  
                return valueProp  
            },  
            set: function(newValue){  
                valueProp = newValue  
                console.log(`${key} es ahora ${valueProp}`)  
            }  
        })  
    }  
}
```

Posteriormente hacemos uso de dicho decorador

```
class Animal{  
    @LogCambios  
    name: string  
}
```

```
var a = new Animal()  
a.name = 'Pipo'
```

Tema 3. Arquitectura de la aplicación

3.1. Introducción y objetivos

El objetivo de este tema es presentarte la estructura básica a partir de la cual se genera la mayoría de las aplicaciones Angular. Aunque dispongamos de cierta flexibilidad a la hora de crear nuestras aplicaciones, necesitamos seguir una serie de normas y disponer de una serie de ficheros específicos para unir todos los elementos que vamos a ir generando.

Dicha estructura ya viene dada a partir de la creación de aplicaciones a través del CLI de Angular por lo que nosotros como desarrolladores solo tenemos que fijarnos en seguir unas buenas prácticas que se asemejen a lo que nos proponen los creadores del framework.

En las nuevas versiones de Angular los componentes son las piezas fundamentales de nuestras creaciones y es por eso por lo que conforman el núcleo central de todas las aplicaciones que vamos a crear.

Tema 3. Arquitectura de la aplicación

3.2. Partes de una aplicación de Angular

Las últimas versiones de Angular han incluido un cambio fundamental en la arquitectura que usamos para poder desarrollar nuestras propias soluciones. Se ha colocado al **componente en el centro de todo**.

Cuando nos planteamos desarrollar cualquier aplicación con Angular debemos tener en cuenta que el resultado final será un puzle conformado por una serie de componentes que serán capaces de comunicarse entre ellos y con el exterior. Es muy importante que mantengamos la independencia de dichos componentes para poder desarrollar así un código que se desacople de manera sencilla del contexto en el que lo creamos.

En versiones anteriores a la 17 observamos que la estructura fundamental de nuestras aplicaciones se asentaba en el **uso de módulos**. Podemos seguir trabajando con este concepto para poder agrupar diferentes funcionalidades y que «trabajen» bajo el mismo nombre. Durante los siguientes temas analizaremos los módulos más importantes que vamos a tener que seguir incluyendo dentro de nuestras aplicaciones.

Los **componentes** son lo que en programación denominamos las vistas. Están representados por clases Typescript, dentro de las cuales encontramos la lógica necesaria para poder crear las diferentes interfaces que componen nuestra aplicación.

Pueden tener atributos tanto de entrada (decorador `@Input`) como de salida (decorador `@Output`). También podemos definir al componente como:

Componente = template + metadatos + clase

Tema 3. Arquitectura de la aplicación

Los **templates** definen la vista de un componente, el HTML y el CSS que va a usar el componente para mostrar el resultado.

Los **decoradores** son funciones que se colocan delante de la definición de una propiedad, clase o función y que ofrecen la posibilidad de configurar el comportamiento de esa clase, propiedad o función a la que decoran. Son los **metadatos de los componentes**.

El **data binding** nos permite pintar o intercambiar datos con el template y la clase del componente, es la forma en la que comunicamos la parte de la lógica con la parte de la vista y podemos mostrar el resultado.

Existen cuatro tipos de **data binding**, los cuales iremos explicando con mayor profundidad durante el curso.

Veamos un poco por encima en qué consisten:

- ▶ **Interpolación:** nos permite pintar, dentro del HTML, cualquier valor que esté definido dentro de una propiedad de clase de nuestro componente. Debemos definir esta propiedad e inicializarla dentro de nuestra clase de componente, de esta forma podremos pintar dicho valor a través de las dobles llaves de esta manera `{{propiedad}}`.
- ▶ **Event binding:** nos permite ejecutar un evento desde el HTML de nuestro componente, de esta forma `(click) = "hacerClick()"`. Dicha función deberá estar dentro del `.ts` del propio componente y se ejecutará siempre y cuando el usuario interaccione con la interfaz, en este caso haciendo click. Nos permite comunicar información desde el HTML hacia el TS.
- ▶ **Property binding:** es la forma que tenemos de asignarle a una propiedad de HTML el valor de una propiedad creada dentro de nuestro componente.

`[src] = " propiedad "`

Tema 3. Arquitectura de la aplicación

Es como realizar una interpolación dentro de un atributo de HTML, lo que permite insertar cualquier sentencia JavaScript válida dentro de los atributos del HTML.

- ▶ **Two-way binding:** permite comunicar el HTML y el TS del componente de forma bidireccional, es decir, si el valor cambia en el HTML, se guarda en el TS y viceversa.

Las **directivas** permiten añadirle un **comportamiento dinámico** al HTML mediante un atributo o selector. Existen dos directivas estructurales, `*ngIf` y `*ngFor`, y otras de atributo que modifican el aspecto visual de un elemento del DOM, estas son `[ngClass]` y `[ngStyle]`. También podemos tener directivas mixtas que tienen parte estructural y parte de atributo, como puede ser `ngSwitch`.

En las nuevas versiones de Angular, aunque podamos seguir utilizando estas directivas, se han incluido herramientas más completas para poder controlar la creación de nuestras diferentes interfaces.

Los **servicios** son clases que permiten comunicar a los componentes entre sí y además obtener datos externos a través de peticiones por HTTP a un servidor. Se encargan de distribuir la información y segmentarla a través de funciones para que puedas pedir cualquier dato que necesiten nuestros componentes.

Los **inyectables**, o lo que se conoce como inyección de dependencias, permiten suministrarle funcionalidades a un componente o a un servicio sin que este tenga que crearlos. Usan el decorador `@Injectable`. Están pensados para proporcionar funcionalidades y modularizar el código.

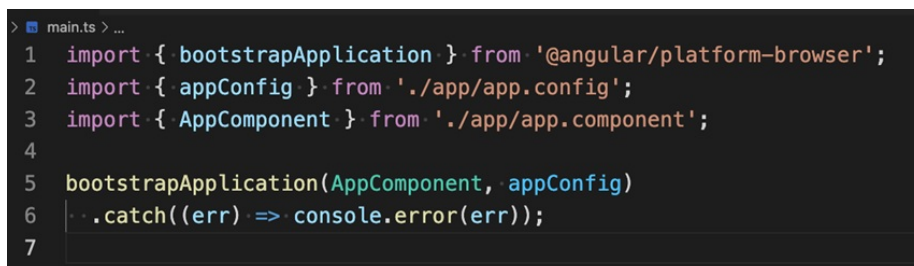
Tema 3. Arquitectura de la aplicación

3.3. Configuración básica

Uno de los cambios fundamentales de las últimas versiones de Angular es la capacidad que tienen ahora los componentes para poder **trabajar de manera independiente**, sin necesidad de estar asociados a un módulo.

Esto hace que, para poder crear una estructura de aplicación sólida y escalable, necesitemos un único componente que actuará como componente principal y a partir del cual colgarán el resto de los componentes. De esta manera estaremos generando una **jerarquía de componentes** que es fundamental conocer para poder establecer los diferentes flujos a través de los cuales movemos la información.

Para entender cómo se inician nuestras aplicaciones debemos revisar el fichero **main.ts**.



```
> main.ts > ...
1 import { bootstrapApplication } from '@angular/platform-browser';
2 import { appConfig } from './app/app.config';
3 import { AppComponent } from './app/app.component';
4
5 bootstrapApplication(AppComponent, appConfig)
6   .catch((err) => console.error(err));
7
```

Figura 9. Fichero main.ts. Fuente: elaboración propia.

Se trata del fichero inicial que se ejecuta cuando ponemos el servidor de nuestro proyecto a funcionar.

Lo más importante de este fichero es la ejecución del método **bootstrapApplication**. Como parámetros le estamos pasando, en primer lugar, la **referencia al componente** que va a actuar como componente **principal** (AppComponent) y, en segundo lugar, un **objeto** en el que tendremos reflejada la **configuración más global de nuestro proyecto** (app.config.ts).

Tema 3. Arquitectura de la aplicación

La **diferencia fundamental** de esta arquitectura con respecto a la arquitectura modular es que, en esta segunda, la activación de todos los recursos contenidos en nuestro proyecto se realizaba en el interior de un módulo.

No es que vayamos a estar constantemente modificando el fichero **app.config.ts**, pero sí es importante que conozcamos su contenido, que sepamos cuándo tenemos que recurrir a él para poder establecer ciertas configuraciones globales dentro de nuestra aplicación.

```
1 import { ApplicationConfig } from '@angular/core';
2 import { provideRouter } from '@angular/router';
3
4 import { routes } from './app.routes';
5
6 export const appConfig: ApplicationConfig = {
7   providers: [provideRouter(routes)]
8 };
9
```

Figura 10. Fuente: elaboración propia.

Como observamos en la imagen, dentro del fichero principal de configuración únicamente definimos e inicializamos la **variable appConfig**. Dentro de las diferentes opciones que incluimos en dicha variable encontraremos, sobre todo, las configuraciones más globales para nuestra aplicación.

En el caso de disponer de un proyecto creado desde cero, la configuración básica se limita a especificar dónde se encuentra el array de rutas que vamos a utilizar para poder saber qué interfaz se activa en función de la URL que dispongamos en el navegador (tema que ya profundizaremos más adelante).

Tema 3. Arquitectura de la aplicación

3.4. Arquitectura por componentes

Angular es un framework para la creación de aplicaciones de cliente basadas en HTML y Javascript (o algún lenguaje que pueda compilarse en Javascript, como Typescript).

La **estructura general** parte de un conjunto de librerías, las obligatorias, que componen el núcleo central de Angular y una multitud de opcionales.

Podemos crear nuestras aplicaciones Angular mediante la creación de plantillas dentro de componentes propios, agregando servicios que manejen la lógica de nuestra aplicación y empaquetando nuestras creaciones en una jerarquía de componentes tal y como hacemos cuando se maqueta una página web normal y corriente.

A partir de ahí, simplemente tendríamos que determinar cuál es el **componente raíz** a partir del cual arrancaría nuestra aplicación.

A la hora de organizar nuestro código es recomendable pensar en la escalabilidad y la limpieza dentro de nuestra aplicación. Son conceptos que nos ayudan a mantener una buena organización y un correcto flujo de trabajo mientras nos encontramos en la etapa de desarrollo. Uno de los conceptos fundamentales que debemos tratar antes de ponernos a desarrollar es **cómo estructuramos los directorios** dentro de nuestra aplicación para que **nuestro trabajo sea óptimo**.

Partimos de la base de que cada proyecto es totalmente diferente a los anteriores y que, dependiendo de nuestra experiencia o nuestra forma de trabajar, podremos implementar una arquitectura u otra.

Existe una vertiente muy extendida en el desarrollo de aplicaciones Angular que **divide el código** generado **por funcionalidades**. Dependiendo de la labor que

Tema 3. Arquitectura de la aplicación

vayan a cumplir nuestros componentes dentro del conjunto de la aplicación final, serán colocados en un directorio u otro.

Seguir esta forma de trabajar nos permite tener localizados en todo momento las partes fundamentales de nuestra aplicación y **facilita el trabajo en equipo sin incidir** en demasiados **conflictos de código** (cada persona o equipo puede encargarse de una funcionalidad distinta).

Durante los próximos temas nuestra tarea será analizar las diferentes herramientas que conforman nuestras aplicaciones y situarlas de manera óptima dentro de los proyectos que vayamos desarrollando.

Tema 3. Arquitectura de la aplicación

3.5. Creación de módulos

Aunque los módulos ya no son una parte esencial de la creación de una aplicación con Angular, sí que podemos seguir utilizándolos para agrupar diferentes funcionalidades, clases o componentes que, por su naturaleza, deban trabajar de manera conjunta.

El comando del **CLI** que vamos a usar ahora para generar un módulo nuevo es “**generate**”. A continuación, tenemos que indicar qué es lo que se quiere generar ya que dicho comando lo usaremos a lo largo del curso para generar componentes, servicios, etc.

El comando que me permite generar un módulo es:

```
ng generate module nombre
```

Una vez lanzado este comando en nuestro proyecto, dentro de la carpeta "src/app" se crea un subdirectorio con el mismo nombre del módulo generado. Dentro encontraremos además el archivo con el código del módulo.

Tema 3. Arquitectura de la aplicación

La **estructura básica de un módulo** la podemos observar en el siguiente código:

```
1  import { NgModule } from '@angular/core';
2  import { CommonModule } from '@angular/common';
3
4  @NgModule({
5    declarations: [],
6    imports: [
7      CommonModule
8    ]
9  })
10 export class PruebaModule { }
11
```

Figura 11. Estructura básica de un módulo. Fuente: elaboración propia.

El decorador **@NgModule** nos permite disponer de una serie de opciones para poder agrupar los diferentes elementos que van a formar parte de este módulo.

La propiedad **declarations** es un array en el que colocaremos estos elementos (componentes, pipes, directivas...) que van a formar parte de este módulo que estamos creando y que, por tanto, podríamos utilizar en un futuro asociados a dicho módulo.

Dentro del array que colocamos como valor de la propiedad **imports** situamos la serie de módulos externos al nuestro que vamos a utilizar en el código que generemos para nuestro módulo. En este caso estamos incorporando el módulo **CommonModule** a los imports con todas las herramientas básicas para el trabajo con Angular.

4.1. Introducción y objetivos

Una de las máximas en Angular es que **todo es un componente**. Representan la manera que tenemos en Angular para definir las unidades que vamos a visualizar en nuestras páginas y que contendrán la lógica de estas. Además, se encargan de definir y ejecutar la lógica mediante la cual gestionaremos los eventos y las acciones externas.

Por norma general, nuestras aplicaciones Angular estarán conformadas por un **árbol de componentes** relacionados entre ellos. Según el equipo de desarrollo de Angular:

«Un componente controla una parte de la pantalla llamada vista y se encarga de definir los diferentes bloques reutilizables por la interfaz de usuario de nuestras aplicaciones».

El objetivo principal que queremos cubrir en este tema es que entendamos cuál es la importancia del desarrollo de componentes en Angular y como plantearnos la creación de un sistema basado en componentes.

4.2. ¿Qué es un componente?

Un componente es una clase a la que le precede un decorador `@Component`. Mediante la configuración de este podemos crear nuevas etiquetas HTML mediante las cuales podemos añadir funcionalidad a nuestras aplicaciones controlando determinadas zonas de pantalla.

Básicamente los componentes en Angular se organizan en una estructura de árbol teniendo un componente principal del cual depende todos los demás.

El trabajo con componentes en Angular dispone de dos partes fundamentales:

- ▶ **La definición del componente como elemento html.** En este momento queda definido cómo vamos a acceder a dicho componente, la plantilla que lo representa y demás elementos relacionados con su aspecto.
- ▶ **El desarrollo de la lógica del componente.** Todos los componentes llevan asociada una clase donde vamos a encontrar las acciones que van a poder llevar a cabo cada uno de los elementos declarados dentro de nuestro Componente.

4.3. Partes de un componente

Antes de ponernos a definir las partes de un componente veamos un componente tipo para poder hacernos una idea de cada una de sus partes.

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-prueba',
  standalone: true,
  imports: [],
  templateUrl: './prueba.component.html',
  styleUrls: ['./prueba.component.css']
})
export class PruebaComponent {

}
```

Las primeras líneas del fichero que representa un componente de Angular están reservadas para importar todas aquellas herramientas de la propia librería o externas que vamos a utilizar dentro del propio fichero.

Según vayamos conociendo o creando nuevas clases relacionadas con Angular y necesitemos usarlas dentro de nuestras clases, debemos agregar los import correspondientes.

En este caso únicamente estamos importando el decorador **@Component** de la librería **@angular/core**, donde encontramos la base para la creación de componentes en Angular. Mediante este decorador vamos a poder configurar todos los metadatos que caracterizan a este componente.

El decorador se debe aplicar sobre la clase donde posteriormente implementaremos las acciones de nuestro componente.

Tema 4. Componentes

Un componente se compone de tres partes:

¿Qué son las anotaciones? Son los metadatos que agregamos a nuestro código a través del decorador `@Component` y que permiten definir ciertas propiedades, las más importantes son:

- ▶ **selector:** se trata de la propiedad que define cómo vamos a hacer referencia a este componente dentro del DOM de nuestra página. El nombre debe ir en minúsculas y separado por guiones en caso de que contenga más de una palabra. Cuando queramos utilizar nuestro componente, lo especificaremos de la siguiente manera:

```
<app-prueba></app-prueba>
```

- ▶ **standalone:** se trata de un valor boolean que nos indica si el componente puede existir por sí mismo sin necesidad de pertenecer a un módulo (`true`) o, por el contrario, para poder usarlo, debemos incluirlo dentro de un módulo (`false`). La configuración por defecto cuando generamos componentes es colocar este valor como `true`, pero no debemos descartar la posibilidad de incluir ciertos componentes dentro de algún módulo en nuestras aplicaciones.
- ▶ **Imports:** esta propiedad está profundamente ligada con la anterior. Si nuestro componente trabaja de manera independiente y no está incluido en ningún módulo, necesitamos incluir, dentro de este array, todas las herramientas (módulos externos, componentes, directivas...) que vamos a incluir en el componente que estamos desarrollando. Al encontrarse aislado del resto, si queremos utilizar cualquier elemento, debemos incluirlo aquí.
- ▶ **template / templateUrl :** con estas propiedades especificamos el contenido que va a renderizar nuestro componente cuando en el DOM situemos el selector que lo define.

Tema 4. Componentes

El primero, define el contenido de manera INLINE, mientras que el segundo nos permite concretar un fichero .html donde encontraremos la plantilla que vamos a renderizar.

- ▶ `styles / styleUrls` : podemos definir los estilos asociados a nuestro componente a través de estas dos propiedades. Mediante la propiedad `style` podemos definir los estilos de nuestro componente en línea.

Debemos trabajar como si estuviésemos generando los estilos para cualquier página, es decir, establecer clases, modificar elementos a través de su identificador o a través de su tag.

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-prueba',
  standalone: true,
  imports: [],
  template: `<h1>Hola Angular</h1>`,
  styles: [`
    h1 { color: #fff; }
    .description {
      font-style: italic;
      color: green;
    }
  `]
})
export class PruebaComponent {
}
```

Podemos incluso dividir los diferentes estilos en cadenas de caracteres contenidas dentro del array de `styles`.

La parte negativa de usar esta nomenclatura es que la definición de los parámetros de nuestro componente se vuelve demasiado compleja e ilegible, lo que entorpece las posibles tareas de mejora y escalabilidad.

Tema 4. Componentes

Mediante la propiedad `styleUrls` podemos permitirnos definir los estilos asociados a un componente en un fichero `.css` externo.

Al ser un array de literales, nos permite definir tantas hojas de estilo como sean necesarias para nuestros componentes.

Aparte de las dos propiedades de estilos que tenemos disponibles en la definición de los metadatos de `@Component`, podemos definir los estilos de nuestros componentes de dos maneras diferentes.

La primera sería agregando la etiqueta `<style>` delante del html que vamos a renderizar con el componente.

De igual manera, podemos asignar el atributo `style` a cualquiera de los elementos que coloquemos dentro de nuestra plantilla.

4.4. Cómo crear un componente

Generalmente, un componente está formado por cuatro archivos, aunque uno de ellos no es obligatorio y se puede omitir.

- ▶ `app.component.html` : contiene el código HTML mediante el cual fabricamos la pantalla (o vista).
- ▶ `app.component.css` : define los estilos que usaremos en la vista del componente.
- ▶ `app.component.ts` : está escrito en TypeScript y es el **controlador** de nuestra aplicación. Es donde está definido toda la lógica de negocio de nuestro componente y donde implementaremos todos los métodos necesarios para que nuestro componente funcione.
- ▶ `app.component.spec.ts` : los ficheros `spec.ts` nos sirven para hacer test unitarios y que nuestro resultado sea más fiable.

Tema 4. Componentes

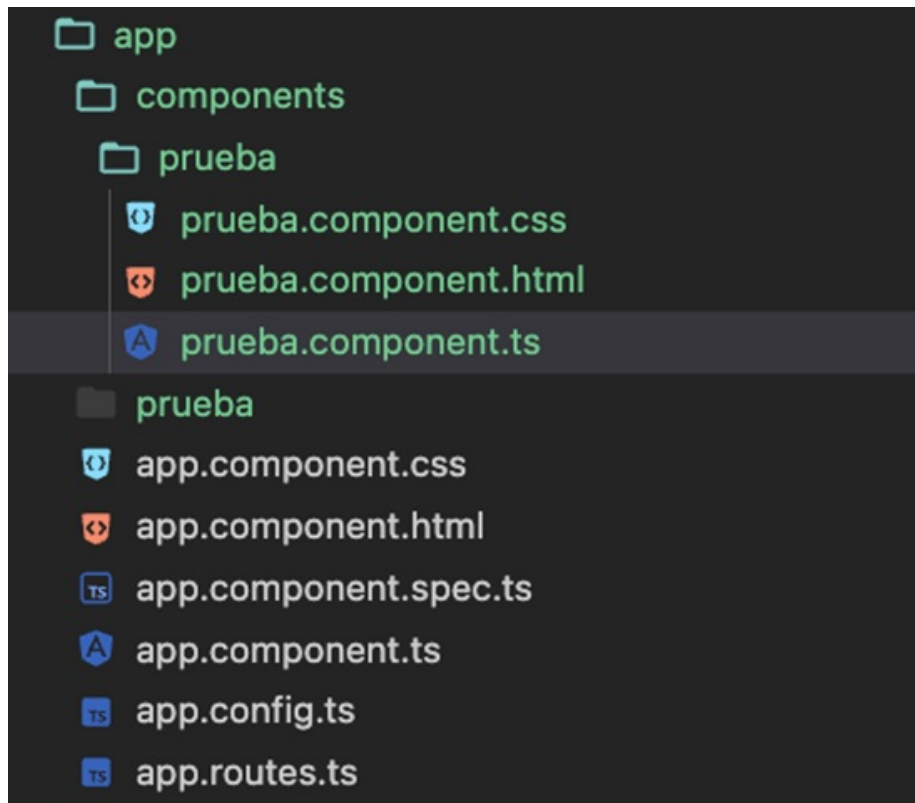


Figura 12. Árbol de carpetas de un componente. Fuente: elaboración propia.

Para crear un componente nos vamos a servir de la herramienta angular CLI que instalamos anteriormente y que, entre otras muchas cosas, nos va a servir para crear todos los elementos que utilizaremos dentro de nuestras aplicaciones Angular.

Para crear un componente escribimos en la terminal elegida el comando:

```
ng generate component components/nombreComponente --skip-tests
```

Tema 4. Componentes

También tienes un formato corto para este script en terminal:

```
ng g c components/nombreComponente
```

El flag `--skip-tests` sirve para evitar la creación del fichero de test en cada component usará siempre que quieres que dicho fichero no aparezca en la carpeta del componente.

4.5. Componente principal y componentes hijos. ¿Cómo se enlazan?

Una de las herramientas más potentes dentro de Angular es la posibilidad de **anidar componentes**.

Para crear una aplicación compleja, podemos implementar los componentes de la manera más sencilla posible para posteriormente, combinarlos entre sí.

Si queremos utilizar un componente dentro de la estructura html de otro, lo podemos hacer incluyendo el selector del componente hijo dentro del espacio donde queramos que se renderice del componente padre

Para que esto sea posible, no debemos olvidar incorporar el componente hijo dentro del array de imports de la configuración del componente padre.

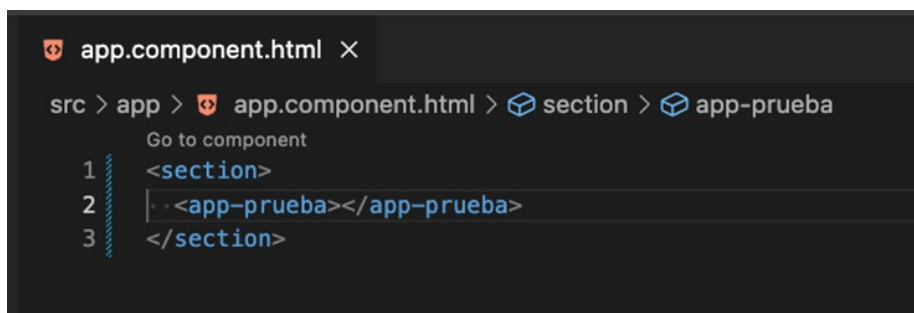
```
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import { PruebaComponent } from '../components/prueba/prueba.component';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet, PruebaComponent],
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  title = 'primeraApp';
}
```

Tema 4. Componentes

Por último, utilizamos el componente hijo dentro de la plantilla del padre. Recordamos que ahora mismo solo tenemos un componente padre que es `app.component.ts`. Pero esto es extrapolable a otros componentes no solo al componente principal.

Enlazar ambos componentes tenemos que poner la etiqueta del componente hijo dentro del fichero `.html` del padre. De esta forma:



```
src > app > app.component.html > section > app-prueba
Go to component
1 <section>
2 <app-prueba></app-prueba>
3 </section>
```

Figura 13. Cómo se enlazan dos componentes. Fuente: elaboración propia.

Esta forma de enlazar componentes dentro de otros componentes. Si ahora intentamos levantar nuestra aplicación para ver el resultado, solo tenemos que hacer el siguiente comando en la terminal.

```
ng serve -o
```

Esto arrancará nuestra terminal y el resultado que veremos será el siguiente, ya que dentro de nuestro componente principal habremos cargado el componente primer-componente visualizándose de la siguiente manera.

Tema 4. Componentes

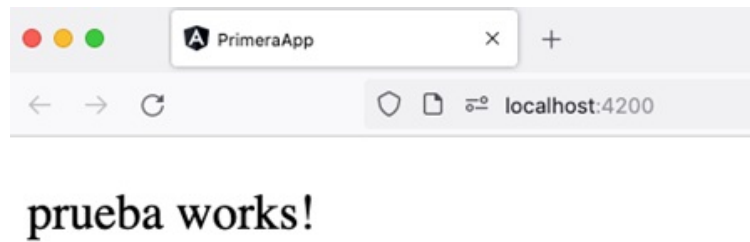


Figura 14. Resultado de la carga del primer componente. Fuente: elaboración propia.

Si modificamos algo del componente, por ejemplo, poner un título en el html del propio componente, de esta forma:

```
<h1>Titulo del primer componente</h1>
<p>Primer parrafo</p>
<p>Segundo parrafo</p>
```

El resultado que saldrá por pantalla tendrá la siguiente pinta:

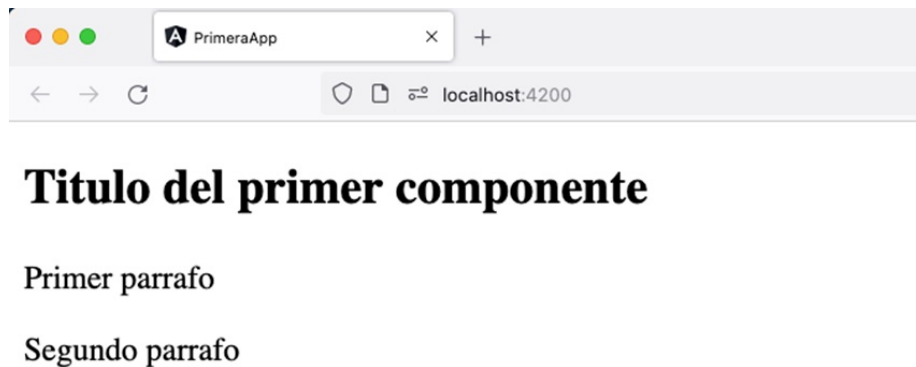


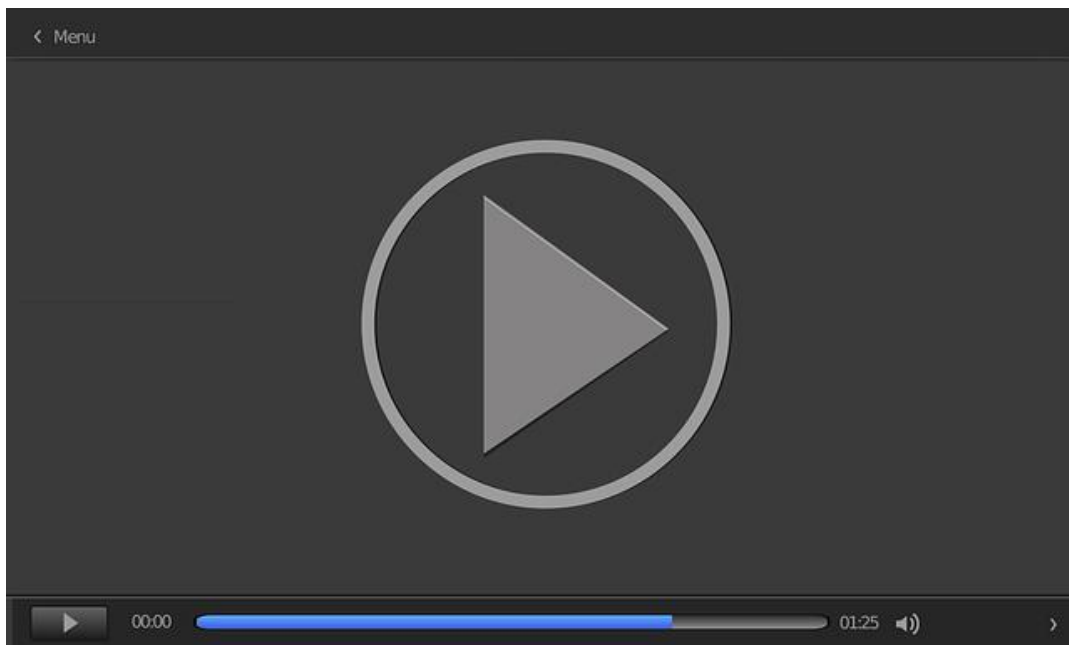
Figura 15. Resultado del navegador. Fuente: elaboración propia.

Tema 4. Componentes

Simplemente hemos incluido algo de maquetación en html dentro del componente sin ninguna funcionalidad, pero el resultado igualmente se visualiza en pantalla.

En el siguiente tema veremos como enlazar el html y css de cada componente a través de su sistema de templating.

En este vídeo, *Creación de componentes*, vamos a ver todos los pasos para la creación de componentes, vamos a crear varios componentes y los vamos a enlazar entre ellos para que se comprenda bien cuál es el proceso y todas sus partes.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=aff00d65-832f-42ba-abac-b127015156b0>

4.6. Funciones de ciclo de vida de un componente

Todos los componentes de Angular disponen de una serie de métodos que podemos implementar y que se ejecutan automáticamente en determinados momentos del ciclo de vida del componente, dándonos la posibilidad de interactuar con cada uno de estos momentos.

De esta manera seremos capaces de realizar diferentes acciones desde la creación del componente hasta su destrucción.

El siguiente gráfico muestra el orden de ejecución de los diferentes métodos del ciclo de vida de un componente.



Figura 16. Orden de ejecución del ciclo de vida de un componente. Fuente: elaboración propia.

Todas ellas tienen un tiempo de ejecución al que podemos engancharnos para ejecutar algunas de nuestras acciones dentro del componente, algunas incluso se solapan y pueden llegar a confundirse por que se ejecutan al mismo tiempo.

Tema 4. Componentes

Vamos a hacer un repaso por las más importantes y que vamos a usar la mayoría de las veces en nuestros desarrollos.

Constructor

Es el método que carga la lógica y he inicializa las propiedades del componente.

`ngOnChanges`

Este método es llamado cuando se aprecia algún cambio en cualquiera de las propiedades que recibe nuestro componente. Veamos un pequeño ejemplo de implementación.

```
ngOnChanges(changes: SimpleChange) {  
  for (let propName in changes) {  
    let chng: any = changes[propName];  
    let cur = JSON.stringify(chng.currentValue);  
    let prev = JSON.stringify(chng.previousValue);  
    console.log(`${propName}: ValorActual = ${cur}, ValorPrevio =  
    ${prev}`);  
  }  
}
```

Estos cambios se aprecian siempre y cuando modifiquemos el valor que le estamos pasando al componente desde el elemento donde lo tenemos definido.

Estos cambios únicamente se observan en un primer nivel.

Es decir, si estamos recibiendo un objeto como valor de un input, las modificaciones que se realicen dentro de los atributos de ese objeto no serán detectadas por el método.

`ngOnInit`

Ocurre tras la primera ejecución de `ngOnChanges`.

Tema 4. Componentes

Es un método que solo ocurre una vez cuando el componente se carga por primera vez. Es un método que está implementado en la interfaz `OnInit` y que inicializa no solo la lógica del componente sino también la parte del HTML.

Implementamos este método por dos razones:

- ▶ Para realizar acciones complejas justo después del constructor.
- ▶ Para configurar el componente después de haber definido los inputs de este.

En el constructor no se deben hacer tareas costosas como la obtención de datos para el componente o la llamada a servicios. Este método sería el lugar idóneo para llevar a cabo estas tareas.

Este método se ejecuta prácticamente a continuación de la creación del componente.

`ngDoCheck`

Hemos visto cómo podemos utilizar el método `ngOnChanges` para recibir los cambios en las diferentes propiedades de nuestros componentes. Estos cambios no se reflejan si las propiedades con las que trabajamos son objetos complejos o arrays. Para ello podemos implementar el método `ngDoCheck`, el cual nos permite trabajar con una comprobación personalizada de los datos.

Este método se lanza después de detectar cualquier tipo de cambio en los datos de nuestro componente. No se debe abusar del uso de este método ya que puede afectar al rendimiento de nuestra aplicación.

`ngAfterContentInit`

Se lanza **después de inicializar el contenido** del componente.

`ngAfterContentChecked`

Tema 4. Componentes

Se ejecuta **tras cada comprobación del contenido** del componente. Comprueba el contenido visualizado en el componente.

`ngAfterViewInit`

Se ejecuta **después de que las vistas del componente se inicialicen** y después de `ngAfterContentChecked`.

`ngAfterViewChecked`

Se ejecuta **después de cada comprobación de la vista de un componente**. Se llama después de `ngAfterViewInit` y de cada `ngAfterContentChecked`.

`ngOnDestroy`

Se trata del método donde colocaremos las **acciones de limpieza de nuestro componente**.

En este momento deberíamos avisar al resto de componentes, si fuera preciso, que el componente actual va a desaparecer. También es el lugar adecuado para liberar recursos utilizados durante el ciclo de vida del componente.

Hay que tener en cuenta las posibles fugas de memoria que pueda provocar nuestro componente.

5.1. Introducción y objetivos

Nos encontramos ante un tema corto, pero bastante visual, ya que en el vamos a intentar cubrir como pasamos de un HTML y un CSS a un componente de angular.

En los primeros temas de HTML y CSS aprendimos todo lo necesario para maquetar de forma estática una web, y con JavaScript le dimos funcionalidad.

En este tema vamos a ver cómo podemos juntar esto en Angular y así poder dar vistosidad a nuestros desarrollos, uniendo nuestra lógica de programación con nuestro HTML y dándole estilos a través de CSS.

Este es el principio de una serie de temas donde vamos a hacer interactuar a nuestra lógica con nuestras vistas, siguiendo el paradigma de programación de MVC, que comentamos al principio en el tema de introducción.

5.2. Sintaxis de plantillas

Hasta el momento, las plantillas que hemos creado para nuestros componentes eran totalmente estáticas.

Angular nos ofrece la posibilidad de ampliar nuestras plantillas e incorporar elementos dinámicos.

Uno de los conceptos más importantes de este **framework** es cómo podemos hacer para enlazar los elementos creados en la clase de nuestro componente con la plantilla que finalmente renderizamos.

Vamos a ver las como comunicamos el archivo `.ts` con el `.html`.

Ya hemos visto que nuestro componente tiene un decorador que asigna, el selector con el que se carga el componente, el CSS que da los estilos al componente y el HTML que renderiza.

En este tema vamos a analizar uno de los primeros elementos del Data Binding, este es uno de los conceptos más importantes en Angular para definir la comunicación entre el componente y el DOM (archivo HTML). Este concepto hace muchos más fácil el pintar los datos variables dentro del HTML simplificando muchísimo el uso del DOM con respecto al uso de JavaScript nativo. El Data Binding son un conjunto de herramientas que Angular nos provee para estos menesteres.

De tal forma que no solo vamos a poder pintar datos en el HTML si no variar estilos y propiedades del DOM de una forma más sencilla.

La comunicación dentro del componente se puede realizar en varios sentidos y dependiendo de este tenemos diferentes tipos de elementos.

Tema 5. Templating

Dentro del Data Binding hay tres tipos diferentes de comunicación entre plantilla, lógica y estilos. Veamos los tres modos y en los próximos temas analizaremos en profundidad cada uno de ellos.

- ▶ One Way Data Binding (del componente al DOM): **Interpolación, Property Binding y Class Binding.**
- ▶ One Way Data Binding (del DOM al componente): **Event Binding y \$event.**
- ▶ Two Way Data Binding (del componente al DOM y viceversa): **FormsModule y [(ngModel)].**

En este primer tema vamos a hablar de la interpolación como la forma más sencilla de visualizar propiedades de un componente en su template. Es la capacidad que tenemos de incluir datos dentro de nuestras plantillas.

Para realizar una interpolación solo necesitamos crear una propiedad dentro de la clase del componente. Este enlace se realiza incluyendo el código dinámico dentro de los símbolos `{{ }}`.

Es la estructura determinada por las llaves podemos incluir cualquier sentencia JavaScript válida excepto:

- ▶ Asignaciones.
- ▶ Creación de nuevas variables.
- ▶ Expresiones encadenadas.
- ▶ Incrementos/decrementos.

Cualquier sentencia que incluyamos dentro de las llaves será ejecutada como si de JavaScript se tratase.

Pero sin duda, el uso más habitual para esta técnica es la de mostrar el valor de las diferentes propiedades contenidas dentro de la clase del componente

El enlace entre la clase y la plantilla es directo y, por lo tanto, a través del nombre de la propiedad, podemos renderizar su valor en la plantilla.

Veamos un ejemplo:

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-prueba',
  standalone: true,
  imports: [],
  templateUrl: './prueba.component.html',
  styleUrls: ['./prueba.component.css']
})
export class PruebaComponent {

  //creamos una propiedad de clase, es como la creación de una variable
  pero sin la palabra let.
  titulo: string;
  //inicializamos en el mismo punto de la creación
  numero: number = 2;

  constructor() {
    //las propiedades hay que inicializarlas y podemos hacerlo en el
    constructor
    o en la propia declaración de la propiedad. Para referirnos a la propiedad
    dentro
    de la clase usamos la palabra this.
    this.titulo = 'Título del Componente'

  }
}
```

En el ejemplo de arriba vemos que hemos creado dos propiedades **título y numero**. Las propiedades se declaran por encima del constructor como si de dos variables globales se tratasen, pero sin usar la palabra `let`, `var`, `const` que usábamos en JavaScript.

Tema 5. Templating

Dichas propiedades aparte de declararla y asignarles un tipo, tenemos la obligación de inicializarlas, y esta tarea la podemos realizar en dos zonas, o en la misma declaración de la propiedad, asignándole un valor por defecto o en el constructor de la clase donde tendremos que usar la palabra reservada **this** para referirnos a la propia propiedad y asignarle un valor.

A través de esta propiedad y la interpolación podemos visualizar el valor de estos datos dentro del HTML de la siguiente forma.

```
<h1>{{ titulo }}</h1>
<p>{{ numero }}</p>
<p>Segundo</p>
```

De esta forma el resultado que aparecerá en nuestra pantalla será el siguiente.

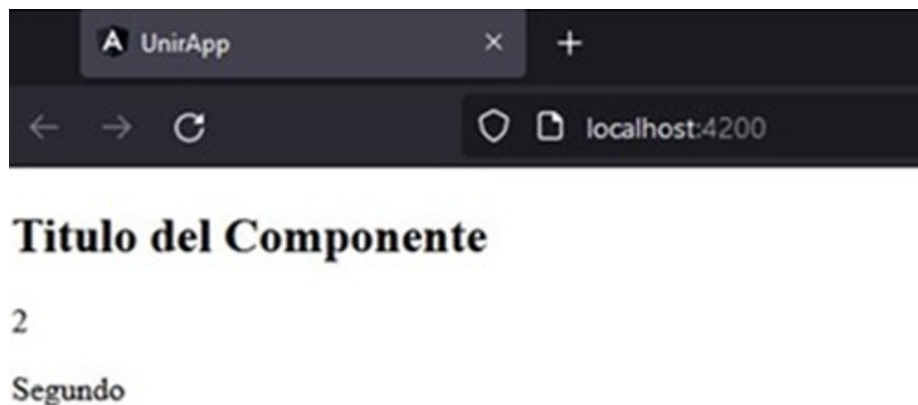


Figura 17. Renderizado de la interpolación de título y número. Fuente: elaboración propia.

Aunque es la forma más sencilla de comunicar valores variables entre la lógica y el html, también podemos hacerlo a través de métodos dentro del componente.

Tema 5. Templating

Podemos declarar una función dentro del componente que devuelva un texto y dicha función pintarla como resultado dentro de nuestro HTML. Veamos su ejecución.

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-prueba',
  standalone: true,
  imports: [],
  templateUrl: './prueba.component.html',
  styleUrls: ['./prueba.component.css']
})
export class PruebaComponent {

  //creamos una propiedad de clase, es como la creación de una variable
  pero sin la palabra let.
  titulo: string;
  //inicializamos en el mismo punto de la creación
  numero: number = 2;

  constructor() {
    //las propiedades hay que inicializarlas y podemos hacerlo en el
    constructor
    o en la propia declaración de la propiedad. Para referirnos a la propiedad
    dentro
    de la clase usamos la palabra this.
    this.titulo = 'Titulo del Componente'
  }

  getParrafo(): string {
    return 'Este parrafo me esta llegando del return de una función dentro
    de mi componente.'
  }
}
```

Luego la pintamos en el HTML del componente de la siguiente forma:

```
<h1>{{ titulo }}</h1>
<p>{{ numero }}</p>
<p>{{ getParrafo() }}</p>
```

La función `getParrafo()` devuelve un texto que se renderiza en la interpolación del HTML dando como resultado lo siguiente.



Figura 18. Renderizado de un return de la función. Fuente: elaboración propia.

Cuando el resultado que queremos pintar tiene etiqueta HTML la cosa cambia un poco por qué no podemos usar la interpolación como tal si no que debemos usar la propiedad `innerHTML` del DOM, aunque en este caso lo haremos de forma un poco distinta a como se hace en JavaScript.

Pongamos un ejemplo, vamos a crear una propiedad y un método que contengan un texto con etiquetas HTML en su interior de la siguiente forma.

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-prueba',
  standalone: true,
  imports: [],
  templateUrl: './prueba.component.html',
  styleUrls: ['./prueba.component.css']
})
export class PruebaComponent {

  parrafo: string = 'En un lugar de la <strong>mancha</strong>'

  constructor() { }

  ngOnInit(): void { }
```

Tema 5. Templating

```
getContenido(): string {  
    return `Este contenido es un template literal y tiene un listado  
        <ul>  
            <li>opción 1</li>  
            <li>opción 2</li>  
        </ul>  
    `;  
}  
}
```

La propiedad solo tiene una etiqueta `strong` y el método tiene un template literal que renderiza un listado no numerado.

Si usásemos una interpolación para pintar estos elementos de esta forma.

```
<p>{{ parrafo }}</p>  
<p>{{ getContenido() }}</p>
```

El resultado obtenido no sería el esperado ya que las etiquetas se verían como si de texto se tratase, obteniendo este resultado.



Figura 19. Renderizado erróneo de contenido HTML dentro de la plantilla. Fuente: elaboración propia.

Como podéis ver las etiquetas forman parte del contenido y no son interpretadas por el navegador de forma correcta.

Tema 5. Templating

Pero ¿cómo podríamos visualizar el contenido de esos elementos dentro del HTML?

Pues usando la propiedad de Angular `[innerHTML]` dentro de la etiqueta del HTML de la siguiente forma.

```
<p [innerHTML]="parrafo"></p>  
<p [innerHTML]="getContenido()"></p>
```

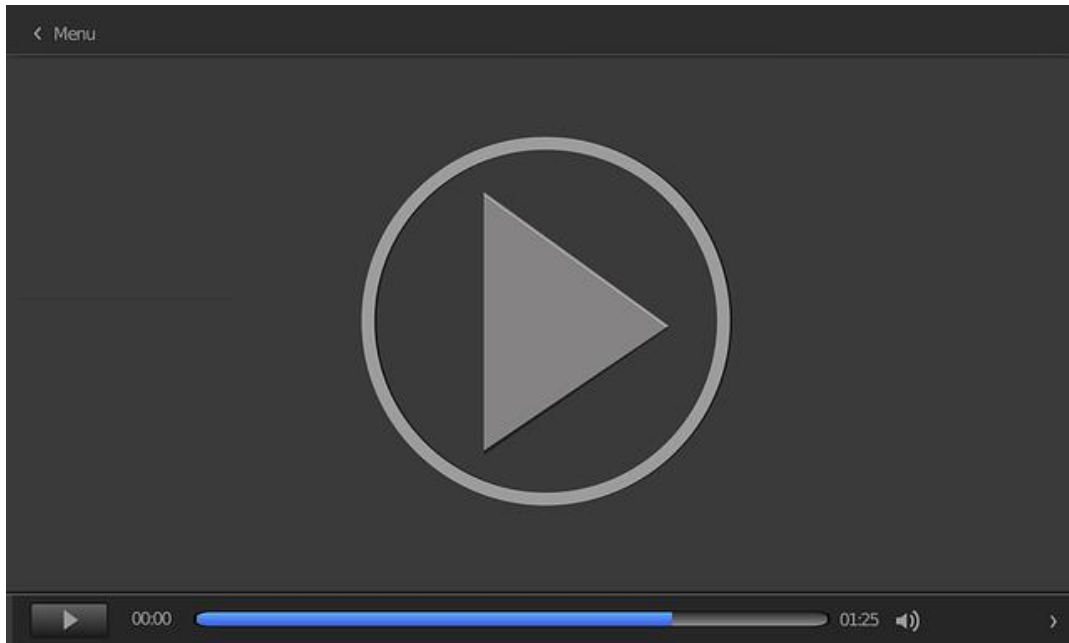
Asignamos como atributo del párrafo dicha propiedad o igualamos al valor que retorna la función para así obtener ahora este resultado. Ahora si visualizándose el contenido HTML de forma correcta.



Figura 20. Renderizado correcto de contenido HTML dentro de la plantilla. Fuente: elaboración propia.

Tema 5. Templating

En el siguiente vídeo, *Estilos de plantilla en Angular*, se explica la creación de un componente donde aplicamos estilos de plantilla para ver cómo generar un interfaz dentro con varios componentes. Solo visual no funcional.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=73f3997e-e631-43a7-a08d-b127015dc02f>

5.3. Hojas de estilos de componente

A la hora de trabajar con estilos de CSS Angular también nos aporta soluciones bastante interesantes.

La primera es que, al tener una hoja de estilos por componente, los estilos de este son independientes de otros componentes, lo cual hace que no nos tengamos que preocupar del nombre de las etiquetas de otro componente porque Angular las hace independientes.

Si queremos que algún estilo afecte a todos los componentes tenemos la hoja de estilos principal `styles.css`.

En esta hoja de estilos podemos cargar los estilos que van a ser comunes y que queremos que afecten a todos los componentes.

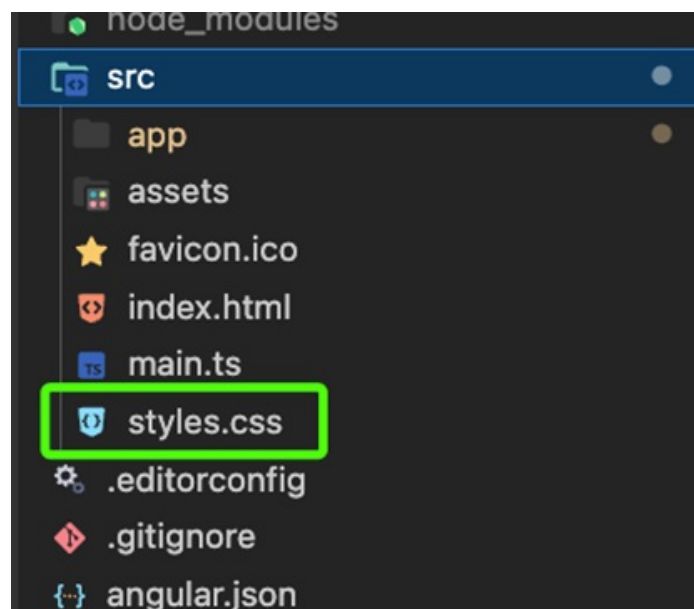


Figura 21. Style.css. Fuente: elaboración propia.

Tema 5. Templating

Por el contrario, si usamos las hojas de estilos propias de cada componente, da igual que estos sean hijos de otros, dichos estilos solo afectaran al componente al que pertenezca esa hoja de estilos, y exclusivamente a él.

Esto hace que no tengamos que estar pensando en si ese nombre de clase que hemos puesto ya lo hemos usado en otro componente y sobre todo permite que si dos desarrolladores trabajan en distintos componentes y usan clases iguales su resultado no se vea afectado por el desarrollo del otro.

Uso de Sass

Dentro de Angular también podemos usar Sass como preprocesador de CSS. Solo hay que tener en cuenta ciertos parámetros.

El primero es que cuando creamos el proyecto debemos marcar que este va a ser con SASS o con SCSS según preferencia. Al elegir una de estas opciones nuestras hojas de estilo CSS desaparecerán para dejar paso a SCSS tanto como hojas generales como dentro de los componentes.

```
Cursos/06_UNIR/Actualización_2024
→ ng new SassExample
? Which stylesheet format would you like to use?
  CSS
> SCSS [ https://sass-lang.com/documentation/syntax#scss ]
  Sass [ https://sass-lang.com/documentation/syntax#the-indented-syntax ]
  Less  [ http://lesscss.org ]
```

Figura 22. Menú de elección de SCSS. Fuente: elaboración propia.

El propio compilador de Angular se encargará de transformarlas a CSS y que podemos ver el resultado en tiempo real dentro de nuestro localhost.

Otro punto importante a la hora de trabajar con SCSS en Angular es que tenemos que configurar el angular.json para decirle donde se van a alojar los ficheros parciales de SCSS dentro de nuestra aplicación. Lo normal para que estén accesibles para todos los componentes que los importen es en la carpeta assets, de la siguiente

forma. Damos de alta justo debajo de script una propiedad `stylePreprocessorOptions` que contiene otro propiedad donde se incluyen la ruta donde van a ser alojados los archivos parciales de CSS, esos que empiezan por guión bajo `_variables.scss`, etc.

```
"styles": [  
  "src/styles.css"  
],  
"scripts": [],  
"stylePreprocessorOptions": {  
  "includePaths": ["assets/partials"]  
}
```

A partir de colocar dentro de esa carpeta todos los archivos parciales de angular, cada componente los podrá importar de forma personal el que necesite en cada momento y de forma independiente.

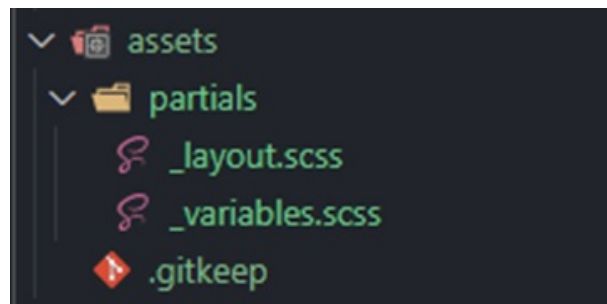


Figura 23. Sistema de carpetaas SCSS dentro de un proyecto. Fuente: elaboración propia.

5.4. Angular signals

A partir de la versión 16 de Angular tenemos disponible la tecnología de **signals** mediante la cual podemos gestionar los diferentes estados dentro de nuestra aplicación. Sería una alternativa para tener en cuenta para gestionar cómo se modifican las propiedades de nuestros componentes que posteriormente visualizaremos dentro de las diferentes vistas asociadas.

A diferencia de las propiedades generadas dentro de la clase de un componente, un signal es un **wrapper en torno a un valor**, el cual es capaz de informar de los cambios ocurridos en dicho valor a todos los «interesados». Es decir, si estamos usando un signal para visualizar un valor dentro de nuestro HTML, cuando lancemos la acción para modificar el valor del signal, éste será capaz de avisar a la vista para poder incluir el nuevo valor.

Para poder visualizar el valor de un signal debemos utilizar su método `get` asociado.

En el siguiente ejemplo podemos ver cómo crear un componente que represente un simple contador.

```
import { Component, signal } from '@angular/core';

@Component({
  selector: 'app-contador',
  standalone: true,
  imports: [],
  templateUrl: './contador.component.html',
  styleUrls: ['./contador.component.css']
})
export class ContadorComponent {

  // Creamos el objeto signal asignándole un valor inicial
  cont = signal(0);
```

```
}
```

Para inicializar el valor correspondiente llamamos a la función **signal** de la librería **@angular/core** pasándole el valor inicial que necesitemos en cada caso.

Para poder recuperar el valor del signal en la vista ejecutamos el método **get** asociado

```
<div>
  <p>Contador: {{cont()}}</p>
</div>
```

Podemos asignar un nuevo valor al signal a través de la función **set**:

```
import { Component, signal } from '@angular/core';

@Component({
  selector: 'app-contador',
  standalone: true,
  imports: [],
  templateUrl: './contador.component.html',
  styleUrls: ['./contador.component.css']
})
export class ContadorComponent {

  cont = signal(0);

  constructor() {
    this.cont.set(10);
  }

}
```

En el momento de realizar la asignación en el constructor, la vista recibirá el aviso de los cambios y actualizará el valor nuevo dentro del espacio que ocupa dentro del HTML.

Si el cambio que necesitamos realizar toma como base el valor anterior, podemos usar el método **update**.

```
import { Component, signal } from '@angular/core';

@Component({
  selector: 'app-contador',
  standalone: true,
  imports: [],
  templateUrl: './contador.component.html',
  styleUrls: ['./contador.component.css']
})
export class ContadorComponent {

  cont = signal(0);

  constructor() {
    setInterval(() => {
      // Modificamos el valor evitando efectos secundarios a partir del
      // valor anterior
      // del signal
      this.cont.update(value => value + 1);
    }, 1000);
  }
}
```

Computed signals

Podemos generar signals que solo sean de lectura y cuyo valor se calcule en función de los cambios que se producen en otro signal que sea de lectura-escritura.

En el ejemplo podemos ver cómo mantener actualizado el número de caracteres de una cadena de caracteres recogida de un campo de texto:

```
import { Component, Signal, WritableSignal, computed, signal } from
 '@angular/core';

@Component({
  selector: 'app-contador',
  standalone: true,
  imports: [],
  templateUrl: './contador.component.html',
  styleUrls: ['./contador.component.css']
})
export class ContadorComponent {
```

```
mensaje: WritableSignal<string> = signal('');
numCaracteres: Signal<number> = computed(() => this.mensaje().length);

onChange($event: any) {
  // Modificando el valor del signal de lectura escritura
  // modificamos también el signal de solo lectura
  this.mensaje.set($event.target.value)
}
}
```

Conclusión

La **gestión de las plantillas** dentro de nuestras aplicaciones de Angular es uno de los conceptos básicos que debemos manejar a la perfección. Conocer todas las posibilidades que tenemos disponibles para comunicar nuestras clases Typescript con las plantillas asociadas facilita muchísimo el trabajo de crear componentes complejos. Para esto es imprescindible manejar la interpolación sencilla y el uso de signals.

La **aplicación de estilos** sobre nuestros componentes también tiene su especial importancia ya que nos permite aislar lo máximo posible los estilos sobre un elemento concreto de nuestra aplicación. El uso de CSS o SASS es muy particular y, sobre todo, depende de la experiencia del equipo de desarrollo.

Tema 6. Data Binding: One-way and Two-ways

6.1. Introducción y objetivos

Este es el segundo tema donde hablamos del Data Binding, como parte importante de la comunicación entre la lógica y la vista del componente.

En el tema anterior hablamos del One Way Data Binding a través de la interpolación de contenidos. En este vamos a continuar con el **One Way Data Binding**, pero esta vez modificando propiedades de las etiquetas HTML a través de propiedades y funciones. Es lo que se llama el Property Binding y me permite modificar aspectos visuales de mis plantillas

Otra parte importante de este tema también vamos a tratar con la comunicación bidireccional entre lógica y HTML a través del **Two Way Data Binding**.

En definitiva, con este tema cerramos el contenido que explica como un componente de angular conecta la lógica de los archivos TypeScript con el HTML y CSS de nuestro componente, consiguiendo así un manejo del DOM que nos facilita muchísimo el trabajo dentro de nuestra aplicación de Angular.

Tema 6. Data Binding: One-way and Two-ways

6.2. Property Binding

Cuando necesitemos **enlazar propiedades** del componente a las propiedades de los elementos del DOM podemos usar Property Binding como alternativa a la interpolación (que también podría usarse en estos casos). Para usarlo pondremos la propiedad del elemento del DOM entre “[...]”, con lo que le asignamos la propiedad del componente.

La asignación seguiría siendo igual que si utilizamos un literal, pero, en este caso, el valor lo obtendremos de evaluar el contenido situado entre comillas. Este contenido podría ser una de las propiedades de nuestra clase, la ejecución de un método o incluso una sentencia Typescript

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-prueba',
  standalone: true,
  imports: [],
  templateUrl: './prueba.component.html',
  styleUrls: ['./prueba.component.css']
})
export class PruebaComponent {

  ruta: string = 'http://www.google.es'

}
```

En el ejemplo que tenemos arriba asignamos a una propiedad de clase ruta el valor de una url con destino a Google. Si queremos asignar dicho valor a una propiedad dentro del HTML, lo haremos de esta forma.

```
<a [href]="ruta">IR A G00GLE</a>
```

De tal forma que cuando renderizamos el resultado ese enlace nos lleva de a Google de manera directa. Esto podría producirse con cualquier propiedad de HTML ya sea

Tema 6. Data Binding: One-way and Two-ways

`src` de imagen, texto alternativo `alt`, `id`, `class`, `title`, en definitiva, cualquier propiedad del HTML puede tener asignada un valor variable en el componente.

Si eliminamos los corchetes, toma el contenido entre comillas como un literal. Con lo que el resultado esperado en el caso del ejemplo anterior es que pusiese dentro del `href="ruta"` en lugar de `href=http://google.es`

Podemos conseguir el mismo efecto con la siguiente sintaxis `href = {{ titulo }}`, aunque desde la documentación oficial de Angular, nos explican que debemos evitar esta forma de aplicarlo e intentar acostumbrarnos a la forma de corchetes ya que el resultado funciona mejor dentro del componente.

Class Binding

Las clases son propiedades también y pueden funcionar de manera muy similar a la de los ejemplos anteriores.

Es decir, podemos tener dentro del `typescript` una propiedad `color` que asigne un valor rojo que corresponda al valor de una clase de `css` de la siguiente manera.

```
import { Component, OnInit } from '@angular/core';

@Component({
  selector: 'app-primer-componente',
  templateUrl: './primer-componentes.component.html',
  styleUrls: ['./primer-componentes.component.css']
})
export class PrimerComponentesComponent implements OnInit {

  ruta: string = 'http://www.google.es'
  color: string = 'rojo'
  constructor() { }
  ngOnInit(): void { }

}
```

Tema 6. Data Binding: One-way and Two-ways

Al llevar esta propiedad a nuestro HTML usaremos los [] como forma de asignar el valor de la propiedad a la clase:

```
<a [href]="ruta" [class]="color">IR A GOOGLE</a>
```

Consiguiendo como resultado los siguiente:

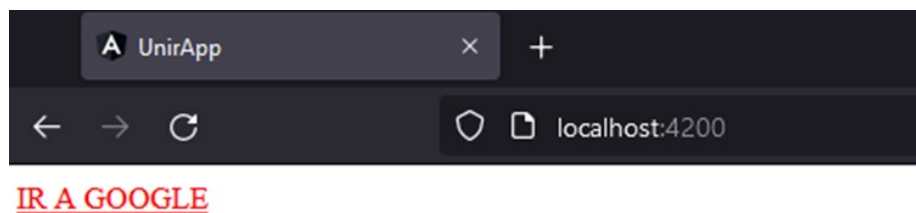


Figura 24. Resultado del Property Binding. Fuente: elaboración propia.

De igual forma, también nos permite especificar una clase de CSS para añadir a un elemento del DOM si una propiedad de un componente es true. Su sintaxis es similar a la de Property binding, con la diferencia de que hay que añadir class delante la clase CSS y que la propiedad del componente debe ser tipo booleano.

Veamos un ejemplo:

```
import { Component, OnInit } from '@angular/core';

@Component({
  selector: 'app-primer-componente',
  templateUrl: './primer-componentes.component.html',
  styleUrls: ['./primer-componentes.component.css']
})
export class PrimerComponentesComponent implements OnInit {

  ruta: string = 'http://www.google.es'
```

Tema 6. Data Binding: One-way and Two-ways

```
color: string = 'rojo'
stock: boolean = true;
constructor() { }

ngOnInit(): void { }

}
```

La propiedad stock es booleana y la vamos a usar para definir que nos pinte una clase rojo si su valor es true de la siguiente forma.

```
<a [href]="ruta" [class.rojo]="stock">IR A GOOGLE</a>

.rojo{
  color:red
}
```

Tema 6. Data Binding: One-way and Two-ways

6.3. Two way date biding: FormsModule y [(ngModel)]. Doble comunicación

Se trata de una de las técnicas más usadas para la recopilación de datos por parte del usuario.

Esta herramienta nos da la posibilidad de enlazar un campo de texto definido dentro de nuestra plantilla con cualquiera de las propiedades del componente, de una manera bidireccional.

Para poder hacer uso de esta característica utilizamos la directiva `ngModel`. Para poder hacer uso de esta directiva, dentro del componente donde vayamos a usarla, debemos importar la librería `FormsModule`.

El fichero con la lógica de nuestro componente quedaría de la siguiente manera:

```
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';

@Component({
  selector: 'app-prueba',
  standalone: true,
  imports: [FormsModule],
  templateUrl: './prueba.component.html',
  styleUrls: ['./prueba.component.css']
})
export class PruebaComponent {

  textoInput: string = 'Texto de prueba';
}
```

Tema 6. Data Binding: One-way and Two-ways

Posteriormente podemos usar la directiva dentro de la plantilla de la siguiente manera:

```
<input [(ngModel)]="textoInput" />  
<h2>{{ textoInput }}</h2>
```

Lo que hacemos con la directiva `[(ngModel)]` es asociar una propiedad de Typescript con la directiva `ngModel` haciendo que así cualquier modificación en el input se guarde en la propiedad y viceversa, obteniendo así una comunicación bidireccional.

En este caso concreto, el valor del campo de texto queda enlazado en ambos sentidos con la propiedad `textoInput` definida dentro de nuestra clase.

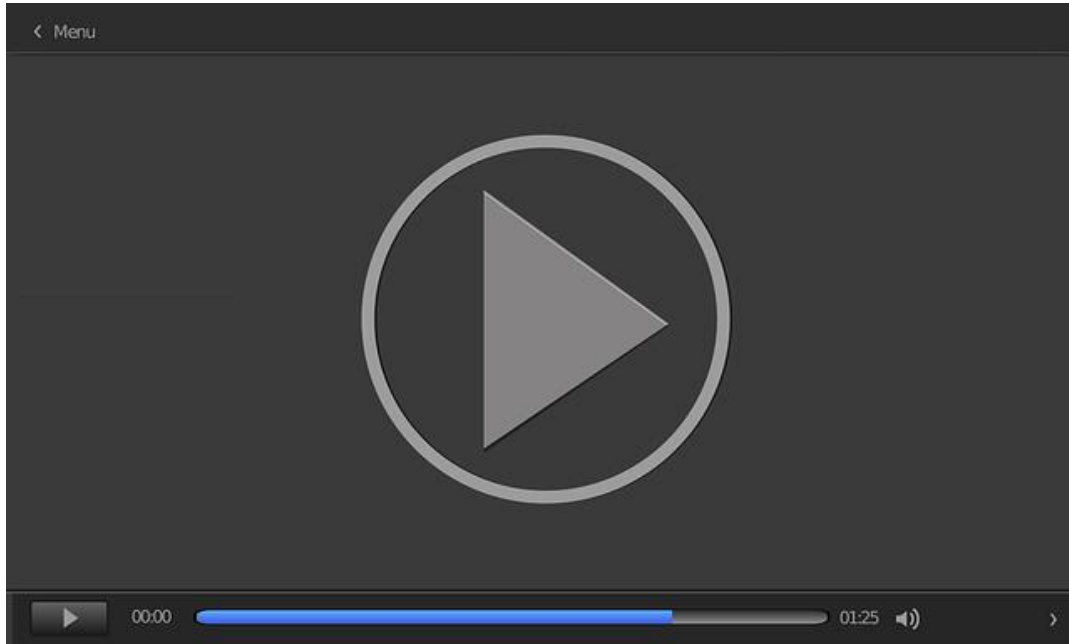
Si modificamos la variable, se modifica el valor del campo de texto y viceversa.

Esto es una forma alternativa a los formularios de angular que nos permite recoger datos por parte del usuario de nuestra aplicación asociando a una propiedad del TypeScript.

En el tema siguiente veremos como a través de eventos podemos usar el Two Way Data Biding para asociar por ejemplo a un objeto el resultado de dos inputs de tal forma que se recoja en cada una de las propiedades del propio objeto los valores del usuario introduce dentro de cada input.

Tema 6. Data Binding: One-way and Two-ways

En este vídeo, *Comunicación entre el TS del componente y el HTML*, se trata la creación de propiedades que me permiten pintar valores dentro del HTML y modificar también atributos de HTML.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=2d4538d9-7222-4dcf-85ab-b12801375e76>

7.1. Introducción y objetivos

El objetivo que buscamos en este tema es interactuar con el usuario de nuestra web. Una página web no es un ente estático si no es un lugar donde el usuario puede interactuar y modificar elementos de la web, ya sea a través de formularios, botones o filtros, el usuario es una parte muy importante del desarrollo de nuestras aplicaciones ya que a través de su interacción recibimos muchísima información que nos ayuda a tomar decisiones.

Esto da lugar a los eventos, y estos están presentes en muchas aplicaciones y de diferentes formas. En JavaScript los manejábamos a través de listeners y Angular nos aporta una forma muy sencilla de manejarlos y recoger los valores que emite nuestra interfaz.

En este tema vamos a ver las diferentes formas que tenemos de capturar los eventos y de mandar información, esta vez, desde el HTML hacia nuestro fichero TypeScript.

7.2. Creación de eventos en Angular

Event Binding nos permite **capturar cualquier evento estándar** del DOM (hacer click en un botón, pasar el ratón por encima de determinado elemento, etc.) y **transmitirlo al componente** para que realice las acciones que convengan. Para hacer uso de Event Binding simplemente pondremos el evento que queramos controlar entre paréntesis, sin el “on” de delante y lo asignaremos a una función del componente.

Podemos utilizar los eventos definidos de manera nativa dentro del navegador o crear los nuestros propios.

Para poder definir qué evento estamos capturando utilizamos los paréntesis para limitarlos.

Vemos un ejemplo para **capturar la acción de click** sobre un botón.

```
<button (click)="hacerClick()">Pulsame</button>
```

Esta captura de código que visualizas es un elemento que se sitúa en el HTML del componente. Entre paréntesis puedes ver el tipo de evento sin la palabra «on» y entre las comillas puedes ver el método de clase que se ejecuta cuando se produce dicho evento.

Este método tiene que estar definido dentro de la clase del componente de la siguiente forma.

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-prueba',
  standalone: true,
  imports: [],
  templateUrl: './prueba.component.html',
  styleUrls: ['./prueba.component.css']
})
```


Tema 7. Event Binding

```
})  
export class PruebaComponent {  
  
  hacerClick() {  
    alert('has hecho click en el boton')  
  }  
  
}
```

Si ejecutas la interfaz lo que vas a poder observar si haces click en el botón es que esta interactúa y te devuelve una respuesta en forma de alerta.

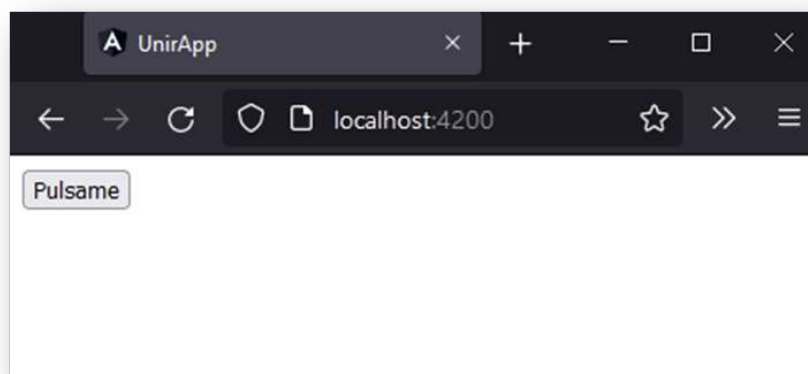


Figura 25. Resultado de la vista de componente antes de hacer click. Fuente: elaboración propia.

Al pulsar en el botón, la interfaz captura el evento, llama al método de la clase y se produce el resultado, en este caso una alerta de JavaScript que nos lanza un mensaje de la siguiente forma.

Tema 7. Event Binding

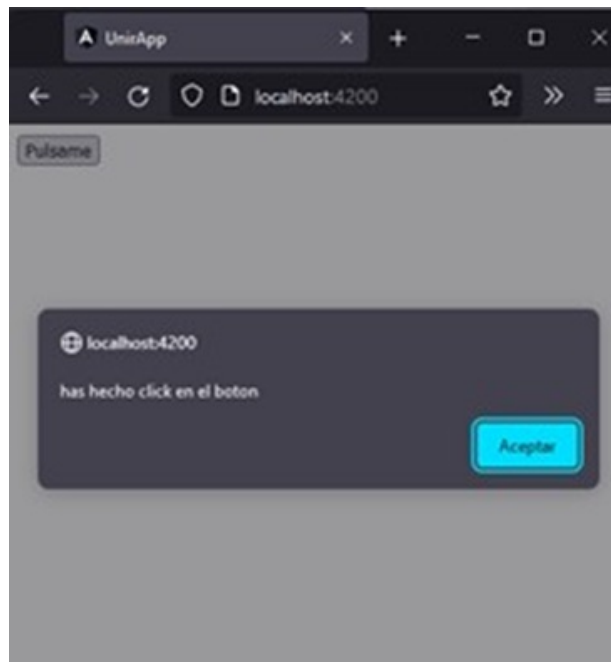


Figura 26. Salida por pantalla de la resolución del evento. Fuente: elaboración propia.

Si unimos la consecución de un evento con lo explicado en el tema anterior podemos asociar el resultado de varios inputs a través del Two Way Data Biding. Os recuerdo los pasos para poder realizar esta técnica:

- ▶ Incluir **FormsModule** dentro de los imports del componente.
- ▶ Crear una propiedad dentro de la clase, esta vez será un objeto JSON.
- ▶ Asociar a cada uno de los inputs una propiedad del objeto.
- ▶ Cuando hagamos **click** en el botón sacaremos por consola el resultado del objeto ya relleno.

Tema 7. Event Binding

Como ya vimos cómo se hacían las importaciones en el tema anterior me voy a centrar solo en la creación y asociación de las propiedades del objeto y en la ejecución del evento.

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-prueba',
  standalone: true,
  imports: [],
  templateUrl: './prueba.component.html',
  styleUrls: ['./prueba.component.css']
})
export class PruebaComponent {

  //creamos e inicializamos la propiedad
  alumno: any = {
    nombre: '',
    edad: 0
  }

}
```

En el fragmento de código anterior vemos como se declara un objeto alumno y lo inicializamos dentro del constructor con las propiedades nombre y edad, con los valores por defecto

En el siguiente fragmento de código vamos a ver como montamos el evento dentro de un botón y como asignamos a cada input los valores de los diferentes elementos.

```
<input type="text" [(ngModel)]="alumno.nombre" placeholder="introduce un
nombre">
<input type="number" [(ngModel)]="alumno.edad" placeholder="introduce una
edad">
<button (click)="capturarDatos()">Pulsame</button>
```

Tema 7. Event Binding

En el Typescript habrá que dar de alta el método de `capturarDatos()` donde simplemente mostraremos los valores del objeto `alumno` por consola.

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-prueba',
  standalone: true,
  imports: [],
  templateUrl: './prueba.component.html',
  styleUrls: ['./prueba.component.css']
})
export class PruebaComponent {

  //creamos e inicializamos la propiedad
  alumno: any = {
    nombre: '',
    edad: 0
  }

  capturarDatos() {
    console.log(this.alumno);
  }

}
```

El resultado que se produce en la consola del navegador es el siguiente.

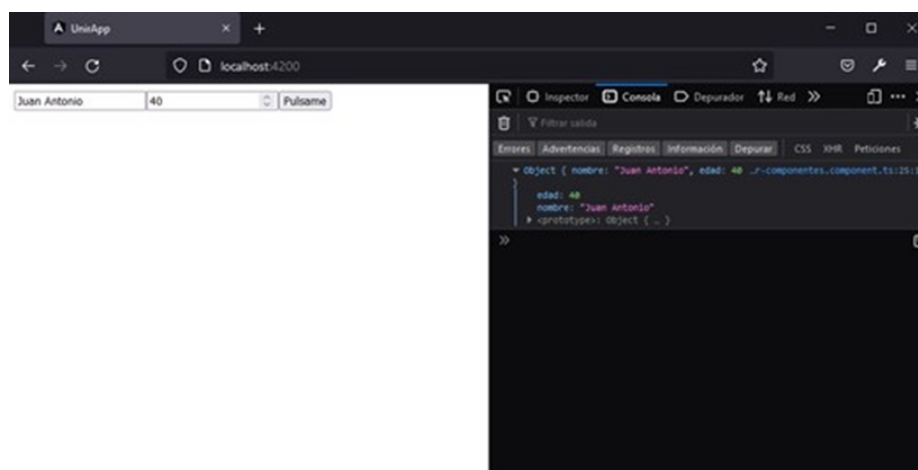


Figura 27. Captura de resultado del evento en consola al hacer click. Fuente: elaboración propia.

Tema 7. Event Binding

Como puedes ver el tomar datos de esta forma combinando las dos técnicas me permite una comunicación bidireccional entre HTML y TypeScript. De esta forma podemos empezar a interactuar entre el usuario y nuestra aplicación.

Por descontado que no hace falta el crear objetos para asignar campos, pero es una buena práctica ya que en un único objeto tengo capturados todos los valores de los campos.

Los eventos que se pueden producir en una aplicación no son solo de click, por supuesto, sino que hay un abanico amplio de evento que pueden resultar interesantes y que se aplican de la misma forma

```
<div (mouseover)=handleOver()">  
<input type="text" (keydown)=handleKey()">  
<form (submit)=handleSubmit()">
```

7.3. \$event

Dentro de los paréntesis del método que lanza el evento podemos insertar cualquier tipo de parámetro, ya sea un string, un número o un objeto completo. Esto podrá cambiar dependiendo de las necesidades que tengamos dentro de nuestro proyecto.

Uno de los argumentos más importantes que podemos pasarle a un evento es `$event`, ya que me permite enviar información del objeto que está lanzando el evento y poder capturar de él, cierta información necesaria para la ejecución de mis acciones.

Depende del evento que estemos capturando podremos recuperar unas propiedades u otras de `$event`.

Volvamos al ejemplo del botón simple y veamos qué posibilidades nos ofrece en este caso pasarle como parámetro al método el `$event`.

```
<button (click)="capturarDatos($event)">Pulsame</button>
```

Como podéis observar en este caso, dentro de los paréntesis de la función tenemos como argumento de nuestra función el `$event` cuando se ejecute la acción descrita, en el método `capturarDatos()` tendremos toda la información que se produce del evento, como por ejemplo la posición en el escenario, el tipo de evento de click, y sobre todo y lo más importante el objeto que lanza el evento, en este caso un `button` de `html`.

```
capturarDatos($event:any) {  
  console.log($event);  
}
```

El resultado que nos arroja la consola de nuestro navegador es el siguiente. Como veis la información enviada es bastante extensa, y entre toda esa información el elemento más importante es el `target` ya que representa el elemento del DOM que tocamos.

Tema 7. Event Binding

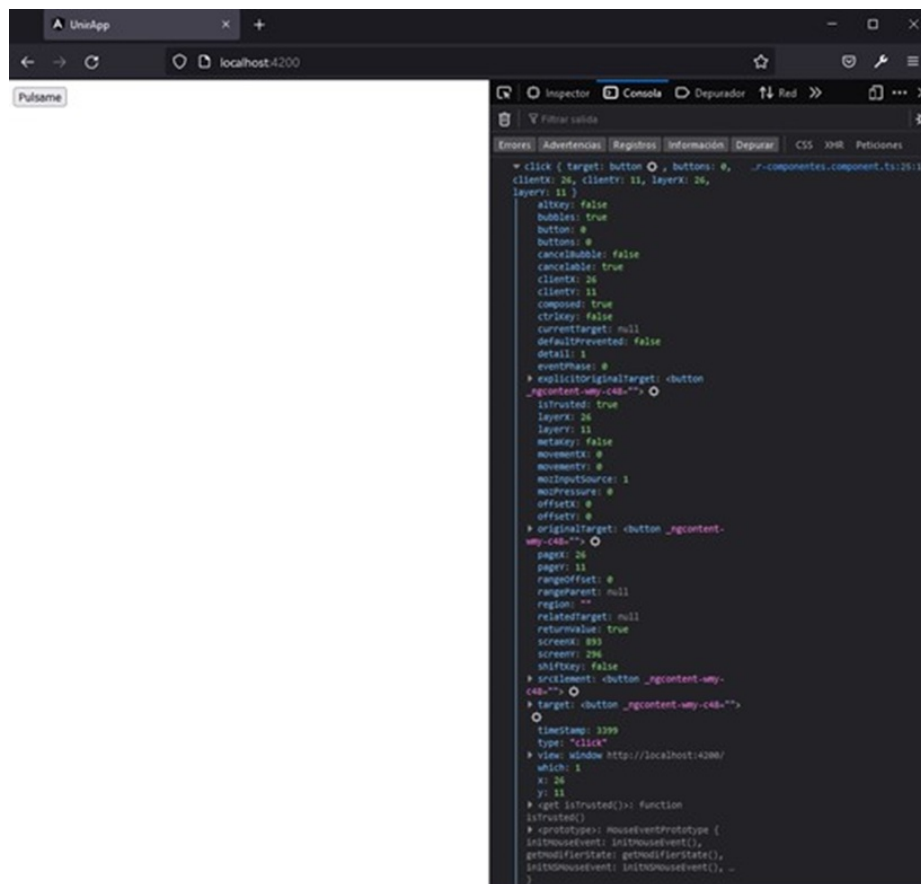


Figura 28. Captura de la salida de consola del navegador. Fuente: elaboración propia.

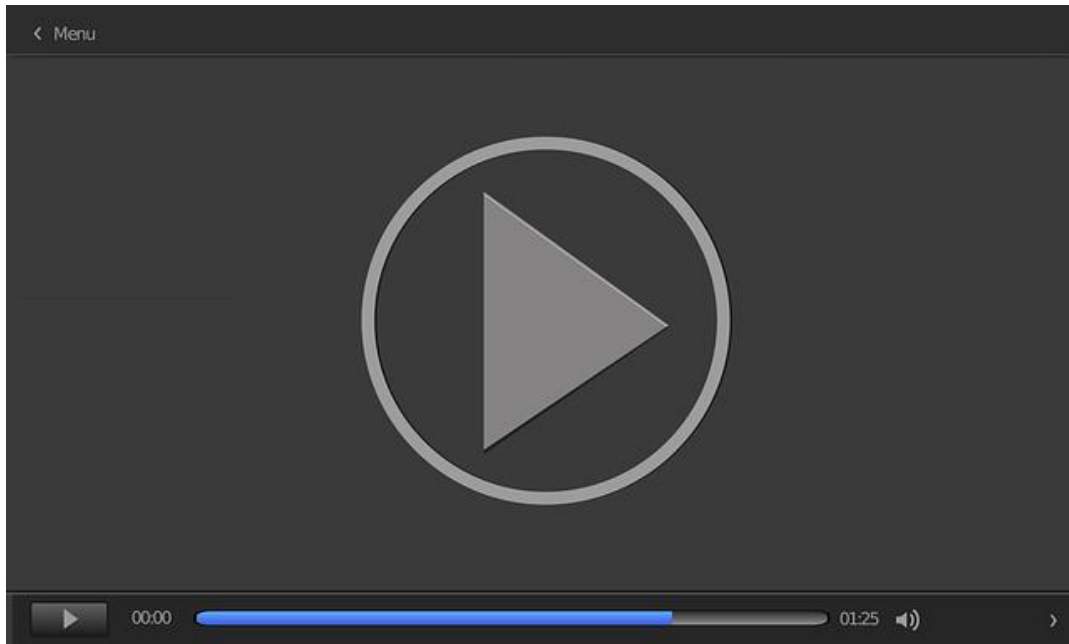
El target al igual que en JavaScript nativo representa el elemento del DOM que ejecuta el evento, y a partir de este elemento podemos capturar una propiedad del objeto, como por ejemplo el id, o el texto del botón con `innerText`, al igual que hacíamos en JavaScript. Por ejemplo `$event` en caso de un evento de teclado me ofrece la posibilidad de sacar el código de tecla para diferenciar entre que tecla del teclado estoy haciendo `keydown`, a través del `$event.keyCode`.

Se debe recordar que a través de un evento podemos cambiar el resultado de cualquier propiedad de nuestro TypeScript lo que me permitiría cambiar una clase o un estilo a través de Property Binding o un texto a través de la interpolación.

Tema 7. Event Binding

Combinando todo lo aprendido hasta ahora, podemos empezar a desarrollar ya aplicaciones un poco más complejas con interacción del usuario.

En este vídeo, *Gestión de Eventos en Angular*, se trata la creación de eventos dentro de Angular para poder interactuar con el usuario.

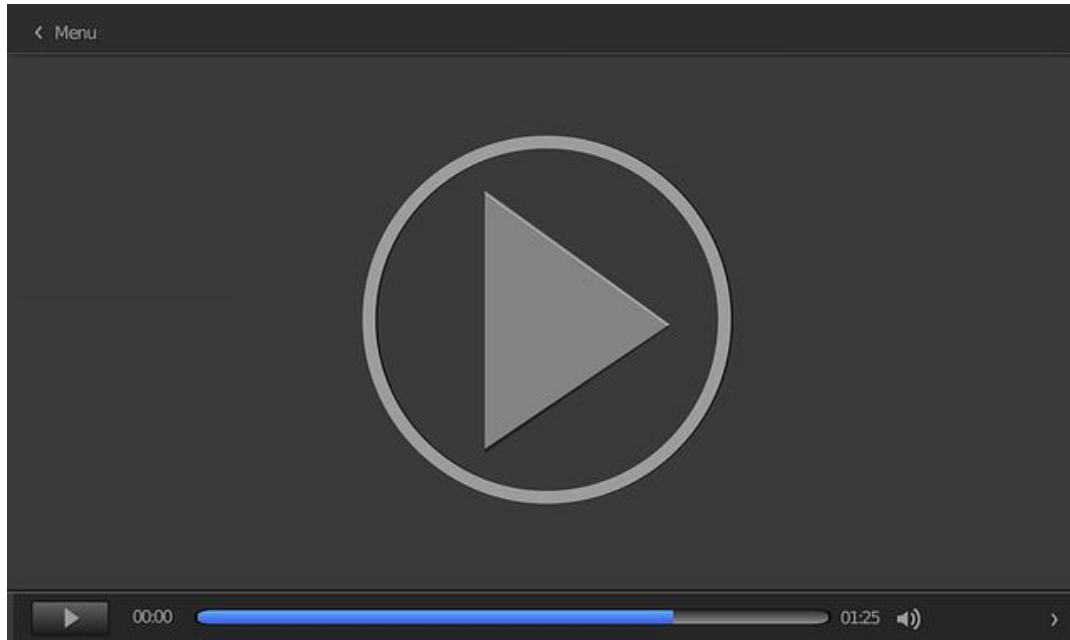


Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=9cdc41ba-d90f-4dc2-8d3c-b128013efaa5>

Tema 7. Event Binding

En el siguiente vídeo, *Two Way Data Biding con Eventos*, se habla de la comunicación bidireccional entre HTML y TS y la interacción con el usuario.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=44a7b734-4141-4d93-85c4-b128014c8316>

8.1. Introducción y objetivos

El objetivo que buscamos en este tema es la comunicación entre componentes. Puede realizarse de diversas maneras (servicios, observables, etc.), pero en este tema, utilizaremos los decoradores `@Input` y `@Output`. El `@Input` es un decorador que van asociado a una propiedad de la clase de nuestro componente, gracias a ellos podemos pasar datos desde un componente padre hacia un componente hijo. Con `@Output`, realizaremos lo contrario. Pasaremos datos desde un hijo hacia su padre. Vamos a ver detenidamente todos los pasos a seguir para usar cada uno de ellos y que particularidades tienen.

El objetivo de este tema es manejarlos con soltura para tener la capacidad de decidir cuando es más recomendable usarlo y cuando, no. Como hemos dicho hay diferentes formas de comunicarse entre componentes y aprender `@Inputs` y `@Outputs` no implica que tengamos que usarlos siempre.

En este tema nos centraremos en manejarlos bien, incluso muchas veces en momentos en los que no sería necesario, pero de esta forma nos ayude a comprender como y cuando es la mejor manera de usarlo.

8.2. Comunicación entre componentes

Dentro de una página web nos podemos encontrar diferentes tipos de componentes que realizan acciones distintas pero que tienen relación con cada uno de los otros componentes que les rodean, en mucho caso estos componentes deben comunicarse algunas cosas y para ello podemos usar diferentes formas.

Como hemos explicado en los objetivos del tema existen muchas formas de comunicar componentes, dependiendo si estos están relacionados entre sí o no.

Durante este curso iremos viendo formas de pasar información entre dos componentes, entre un componente padre y un componente hijo, entre el componente y la BBDD a través del servicio. Es decir que gran parte del acceso a los datos que tienen los componentes la obtienen de la comunicación que puede existir entre los diferentes métodos de comunicarse.

Por matizar un poco más esto nos podemos encontrar con los siguientes casos.

Un componente quiere pasarle información a otro componente que este contenido en él. Es decir, un componente padre quiere comunicar algo a su hijo. Es caso es bastante común dentro de la comunicación entre componentes. En este caso realizaremos la comunicación mediante un @Input .

Tema 8. Input y Output

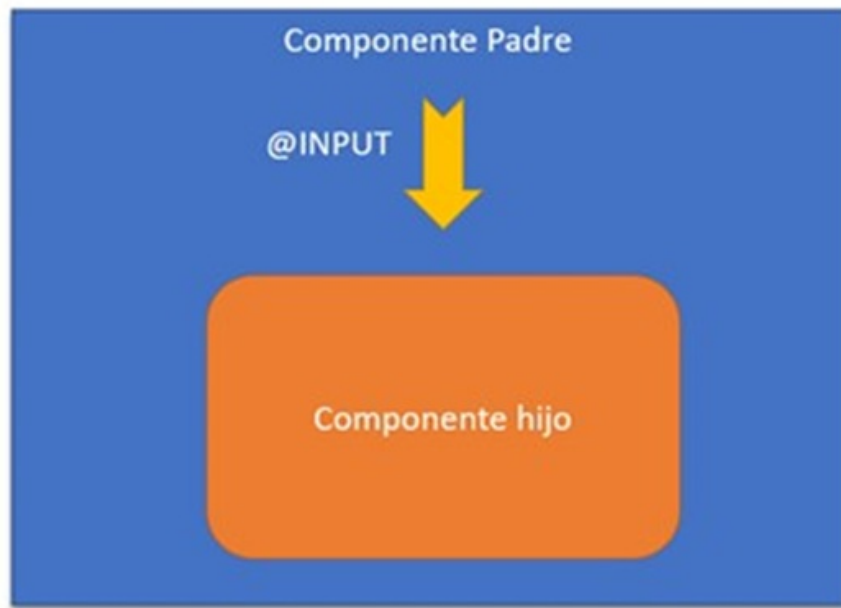


Figura 29. Explicación gráfica de un input. Fuente: elaboración propia.

Un componente que este contenido dentro de otro quiere comunicarle algo a su contenedor. Es decir, un componente **hijo** quiere comunicar algo a un componente **padre**. En este caso la comunicación se debe realizar a través de un evento personalizado o @Output .

Tema 8. Input y Output



Figura 30. Explicación gráfica del output. Fuente: elaboración propia.

Dos componentes que no están relacionados entre sí quieren transmitirse información. En este caso al no haber ningún tipo de relación, directa, entre ellos hay que encontrar un tercer elemento que me permita comunicarme parcialmente con cada uno de los componentes, para ello vamos a usar diferentes técnicas dependiendo de donde se encuentre los datos. Aunque la más común y que veremos más adelante en el curso es la comunicación usando un servicio.

Más adelante el servicio también nos aportará como funcionalidad la posibilidad de hacer consultas a la BBDD externa a la aplicación y poder traer datos para luego enviárselos a los otros componentes de la aplicación. Sería como el director de juego o base en baloncesto, que reparte el juego entre el resto de sus compañeros.

Tema 8. Input y Output

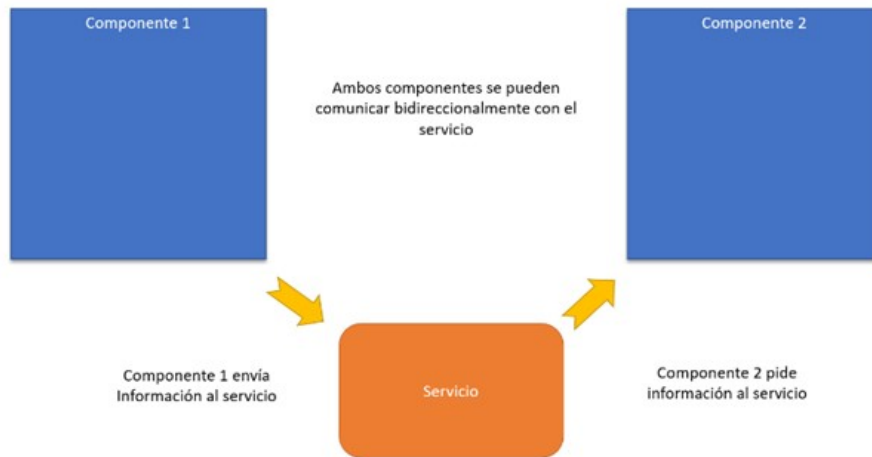


Figura 31. Comunicación a través del servicio. Fuente: elaboración propia.

En este tema nos vamos a centrar exclusivamente en los dos primeros, y más adelante le daremos al servicio un tema propio donde hablaremos de todas sus particularidades.

8.3. Decorador @Input(). Del padre al hijo

Un decorador @input, como ya hemos explicado antes, nos permite comunicar un padre con su hijo y que este le pueda enviar diferentes valores al hijo para cambiar su estado, o simplemente pintar un resultado diferente.

Lo primero que tenemos que hacer para usar el Input es importarlo en el componente que vayamos a usarlo de la siguiente forma.

```
import { Component, Input } from '@angular/core';
```

El siguiente paso es decorar la propiedad con la que vamos a trabajar, para ello como con cualquier decorador le ponemos la función decoradora por delante de la propiedad la cual queremos convertir en Input. Es importante y TypeScript nos avisará de ello, que le pongamos el tipo del dato que va a almacenar ya sea un número o un string, etc. También si TypeScript está en modo estricto nos obligará a inicializar el valor. Esto como cualquier propiedad podréis hacerlo en la misma línea de la declaración o dentro del método constructor de la clase.

```
import { Component, Input } from '@angular/core';
```

```
@Component({
  selector: 'app-prueba',
  standalone: true,
  imports: [],
  templateUrl: './prueba.component.html',
  styleUrls: ['./prueba.component.css']
})
export class PruebaComponent {

  @Input() texto: string = "";

}
```

Tema 8. Input y Output

Ahora tenemos que llenar esa propiedad con un dato, y este se lo vamos a pasar a través del selector que carga el componente dentro de otro, ya sea el principal como otro cualquiera.

```
<section>
  <app-prueba [texto]='Una cadena de texto que viene del padre'></app-
prueba>
</section>
```

Como podéis ver al crear una propiedad texto dentro de la etiqueta del componente, está la ponemos entre corchetes, para que admita el uso de sentencias JavaScript validas como valor del input, en este caso le hemos pasado una cadena de texto, de ahí que el texto este entre comillas simples.

Tal y como veíamos anteriormente, este tipo de atributos también pueden recoger su valor evaluando el contenido entre comillas, mediante la nomenclatura de corchetes.

Podemos definir un alias dentro de la llamada al decorador para especificar un nombre diferente para el atributo que usaremos en la declaración del tag

```
//Creacion de un input
@Input('miTexto') texto: string = "";
```

Si usamos ese alias tenemos que cambiar el valor del input en la etiqueta de creación del componente.

```
<section>
  <app-prueba [miTexto]='Una cadena de texto que viene del padre'></app-
prueba>
</section>
```

Una vez este cargado todo el componente podremos disponer de este input que al ser una propiedad de la clase puede renderizarse perfectamente dentro del HTML del propio componente.

Tema 8. Input y Output

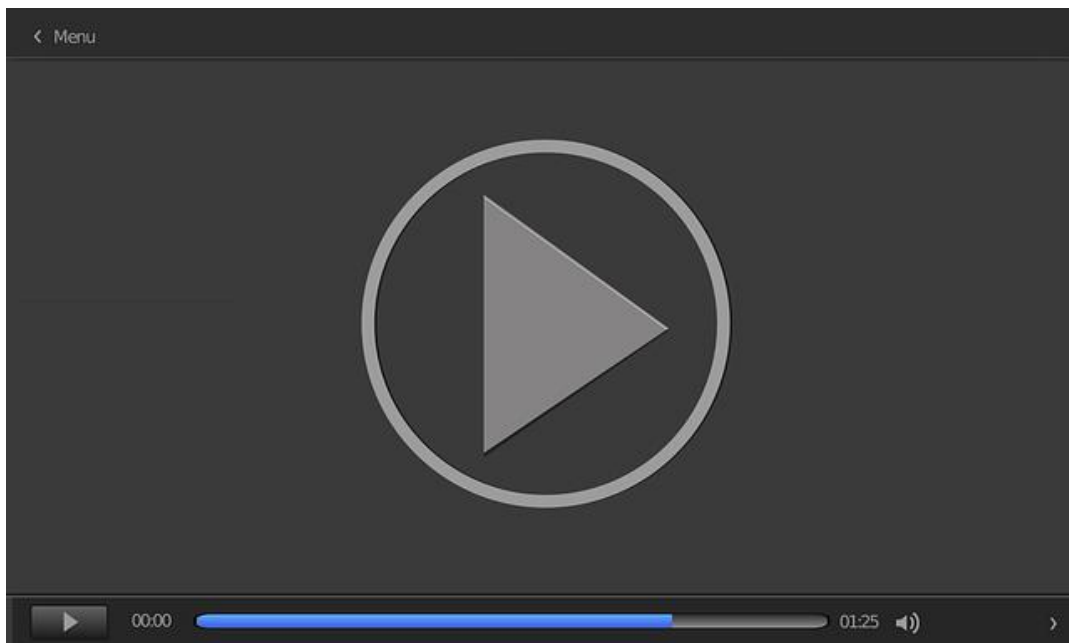
Si os acordáis de las funciones de ciclo de vida de los componentes lo input y los outputs se cargan con el `ngOnChanges` o con el `ngOnInit`.

```
//OJO DENTRO DEL HTML DEL COMPONENTE NO USAMOS EL  
//ALIAS SINO EL NOMBRE DE LA PROPIEDAD  
<p>{{texto}}</p>
```

Visualizar el resultado por el navegador, podréis comprobar que os sale el texto que habríais introducido dentro del del input.

Recordar: Ese texto puede ser una variable o constante, así como una cualquiera propiedad que tenga el componente padre.

En este vídeo, *Inputs en Angular*, se trata la comunicación del padre al hijo a través de inputs.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=29adcdd6-aea2-46f7-9329-b12801522da2>

8.4. Decorador @Output(). Del hijo al padre

Este tipo de decorador lo usamos para pasar información del hijo al padre, el padre esperará un aviso o evento por parte del hijo con el que gestionará la información que este le envía.

Para usar un decorador output al igual que el input hay que importarla en cada componente donde se vaya a usar de la siguiente forma.

```
import { Component, EventEmitter, OnInit, Output } from '@angular/core';
```

También debemos importar la librería EventEmitter que es la que me ofrece la posibilidad de emitir un evento personalizado al padre.

Creemos la propiedad que hacer de output y la inicializamos.

```
import { Component, EventEmitter, Input, Output } from '@angular/core';

@Component({
  selector: 'app-prueba',
  standalone: true,
  imports: [],
  templateUrl: './prueba.component.html',
  styleUrls: ['./prueba.component.css']
})
export class PruebaComponent {

  @Output() delete = new EventEmitter()
}
```

Ahora nos toca crear una función que nos permite llenar el output y emitirle el resultado al padre para que este actúe en consecuencia

Tema 8. Input y Output

Por ejemplo, podemos crear en el HTML un evento que nos envíe un número y este se lo comuniquemos al padre, en el evento onDelete vamos a enviar como parámetro un número y este tenemos que hacérselo llegar al padre para que lo saque por consola.

Primer paso crear el evento y recoger el valor que nos envía.

```
<button (click)="onDelete(5)">Enviar numero</button>
```

Este evento se está produciendo en el HTML del componente con lo que tendremos que crear una función onDelete() en el TypeScript de dicho componente de la siguiente manera:

```
import { Component, EventEmitter, Input, Output } from '@angular/core';

@Component({
  selector: 'app-prueba',
  standalone: true,
  imports: [],
  templateUrl: './prueba.component.html',
  styleUrls: ['./prueba.component.css']
})
export class PruebaComponent {

  @Output() delete = new EventEmitter()

  onDelete(numero: number) {
    this.delete.emit(numero)
  }
}
```

Al recoger el parámetro número del propio evento este se emite hacia al padre creando un evento personalizado dentro de la etiqueta que carga el propio componente.

Tema 8. Input y Output

En el HTML del componente padre la etiqueta del componente antes mencionado tendrá un evento personalizado que se llamará exactamente igual que el nombre del output declarado. Este evento llamará a una función declarada dentro del padre.

```
<section>
  <app-prueba (delete)="handleDeleteFather($event)"></app-prueba>
</section>
```

Este método o evento personalizado recibe un `$event` que en este caso representa al valor del número que le estamos pasando por parámetro. Este valor podremos recogerlo a través de la función `handleDeleteFather($event)` que tendremos que crear dentro del componente padre.

```
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import { PruebaComponent } from '../components/prueba/prueba.component';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet, PruebaComponent],
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {

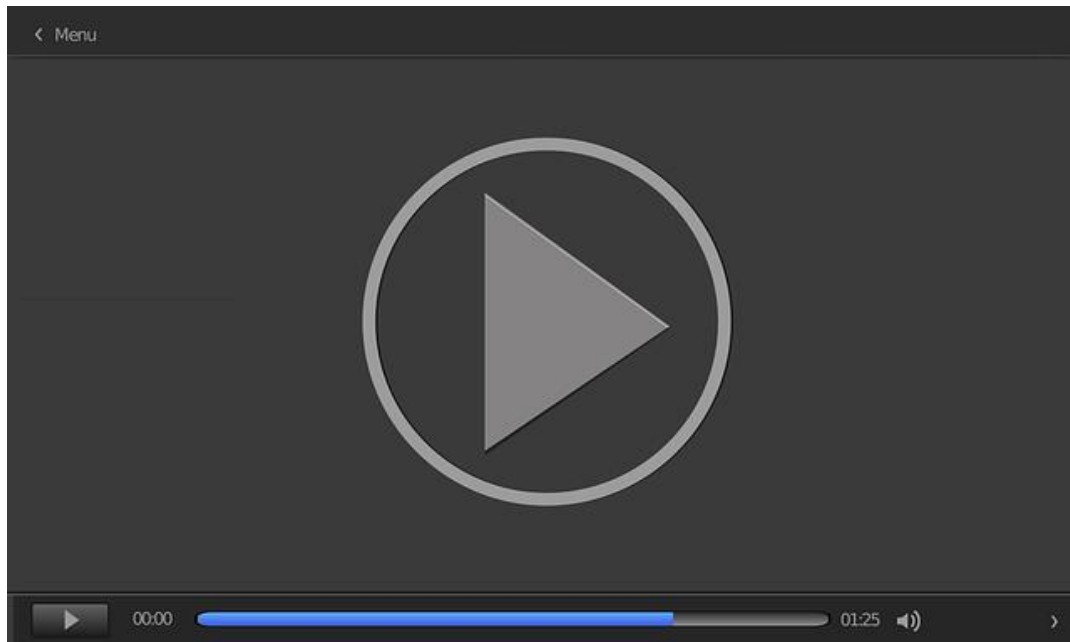
  handleDeleteFather($event: any) {
    console.log($event)
  }

}
```

Cerrando de esta manera el ciclo en el cual el componente hijo le manda un valor al padre y este es capaz de recogerlo a través de un componente personalizado.

Tema 8. Input y Output

En este vídeo, *Outputs en Angular*, se trata la comunicación del hijo al padre a través de outputs.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=c5b0e6a8-365c-42d0-9d90-b1280169d9b5>

9.1. Introducción y objetivos

Con la gestión de la estructura de nuestras plantillas de manera dinámica alcanzamos uno de los puntos fundamentales del aprendizaje de Angular.

Desde la versión 17 del framework tenemos disponibles una **serie de herramientas** que nos van a permitir reorganizar los bloques de código HTML con los que vamos a crear nuestras interfaces de manera dinámica y de una forma mucho más simple que como veníamos haciendo hasta ahora.

Las posibilidades que nos ofrece esta serie de herramientas hacen que nuestro HTML deje de ser completamente estático y se vaya modelando en función de los datos disponibles en nuestros diferentes componentes, así como los que podamos recibir desde el exterior.

A partir de este momento cobra mucho más sentido el hecho de crear aplicaciones basadas totalmente en componentes reutilizables.

9.2. Bloque condicional con @if

Una de las acciones más habituales que vamos a necesitar como desarrolladores es la de mostrar u ocultar el contenido de cierta parte de nuestras plantillas en función de ciertas condiciones.

Estas condiciones van a permitirnos construir la interfaz que le mostraremos al usuario y además van a poder ser modificadas a través de los eventos que active el propio usuario.

La utilización de este bloque nos inhibe de tener que modificar los estilos de un componente concreto para mostrar u ocultar su contenido.

El uso del **bloque @if** es similar al que le damos en Javascript. En el siguiente ejemplo vemos cómo podemos mostrar u ocultar elementos de un componente a partir de una condición.

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-prueba',
  standalone: true,
  imports: [],
  templateUrl: './prueba.component.html',
  styleUrls: ['./prueba.component.css']
})
export class PruebaComponent {

  edad: number = 23;

}
```

Tema 9. Control Flow

Dentro del HTML del componente podemos utilizar la propiedad `edad` para crear nuestra condición.

```
<section>
  @if(edad > 18){
    <p>Eres mayor de edad</p>
  }
</section>
```

El párrafo contenido dentro del bloque condicional se muestra o se oculta dependiendo de si se cumple o no la operación lógica que hemos especificado.

Es **muy importante** que, a diferencia de la utilización de estilos CSS para ocultar ciertos elementos, sepamos que, con el bloque `@if`, el elemento sobre el cual estamos actuando **desaparece por completo del HTML**. Angular es capaz de saber, al analizar la condición, cómo y cuándo debe aparecer dicho elemento.

¿Por qué eliminamos el contenido en vez de ocultarlo?

Podríamos realizar el mismo trabajo que hace este bloque con la aplicación de ciertos estilos CSS que nos permiten mostrar/ocultar el HTML. Con `@if` lo eliminamos directamente.

El comportamiento con CSS es el siguiente:

- ▶ Cuando ocultamos un componente o parte de nuestra página, su comportamiento sigue activo.
- ▶ Sigue escuchando los posibles eventos que reciba la aplicación.
- ▶ Angular seguirá escuchando los posibles cambios que lo afecten y seguirá enviando los datos que se modifiquen.
- ▶ Sigue usando recursos de nuestra aplicación.
- ▶ Por otro lado, el cambio de oculto a no oculto es muy rápido.

El trabajo con `@if` es diferente:

- ▶ El ocultar bloques de código afecta a su rendimiento, ya que se eliminan del DOM.
- ▶ Deja de recibir eventos.
- ▶ El componente es destruido y se ejecutan los métodos finales del ciclo de vida del componente.

En este caso, se emplean muchos recursos para regenerar el componente.

Junto con el bloque `@if` tenemos también la posibilidad de establecer qué elementos se van a visualizar en caso de que la condición sea negativa. Esta acción la logramos a través del bloque `@else`.

```
<section>
  @if(edad > 18){
    <p>Eres mayor de edad</p>
  }@else {
    <p>NO eres mayor de edad</p>
  }
</section>
```

En el ejemplo anterior siempre disponemos de un párrafo con la información. Lo que cambia dependiendo de la operación condicional es el contenido de este.

Dentro de estos bloques, como ejemplo, estamos creando párrafos simples, pero podríamos incluir cualquier bloque HTML válido o incluso componentes personalizados un poco más complejos. No existe ningún tipo de limitación a la hora de establecer los bloques contenidos en una expresión condicional.

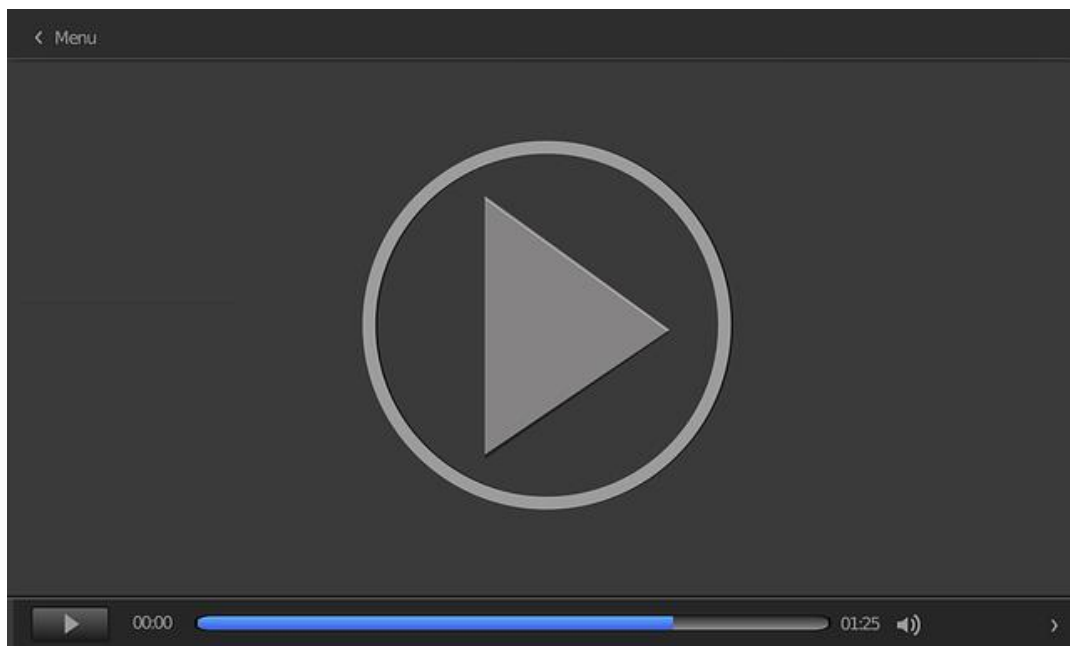
Completando el concepto de condicional semejante al que ya conocemos de Javascript, podemos incluir el uso de `@else if`. Vemos un ejemplo:

```
<section>
  @if (edad > 18) {
    <p>Eres mayor de edad</p>
```

Tema 9. Control Flow

```
} @else if (edad < 18) {  
    <p>Eres menor de edad</p>  
} @else {  
    <p>Tienes justo 18 años</p>  
}  
</section>
```

En este vídeo, *Bloque condicional @if*, veremos cómo se usa el bloque condicional @if.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=29449bca-1e8f-4533-b810-b1280170b791>

9.3. Bloque de repetición con @for

Abandonar el uso estático de los bloques HTML dentro de las plantillas de nuestros componentes es fundamental para agilizar la maquetación en nuestras aplicaciones. El bloque **@for** nos permite, a partir de una colección de elementos, repetir el bloque HTML que seleccionemos tantas veces como elementos tengamos en dicha colección.

Su uso es muy parecido al del **bucle for...of de Javascript**.

Analizaremos su uso a través de un ejemplo. Dentro de la clase del componente creamos e inicializamos un array con una serie de datos.

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-prueba',
  standalone: true,
  imports: [],
  templateUrl: './prueba.component.html',
  styleUrls: ['./prueba.component.css']
})
export class PruebaComponent {

  estudiantes: { id: number, name: string }[] = [
    { id: 1, name: 'Luis García' },
    { id: 2, name: 'Inés Hernández' },
    { id: 3, name: 'Rocío Martín' },
    { id: 4, name: 'Ramón Romero' }
  ]
}
```

La propiedad «estudiantes» la podemos utilizar dentro del HTML del componente para repetir cierto bloque de código:

```
<ul>
  @for (est of estudiantes; track est.id) {
    <li>{{ est.name }}</li>
  }
</ul>
```

Dentro del bucle estamos recorriendo el array «estudiantes» y en cada una de las iteraciones recibimos el valor correspondiente dentro de la variable **est**. Esta variable estará disponible únicamente dentro del ámbito del bucle.

Además, debemos incorporar de manera obligatoria un **valor para la propiedad track** del bloque @for. El valor que asignemos aquí debe ser un **identificador único** que le permita a Angular diferenciar cada una de las iteraciones del bloque. En caso de que el array se modifique, el framework será capaz de identificar qué elemento o elementos HTML son los que debe modificar, con lo cual dejará a los demás tal y como estaban.

Variables asociadas al bloque @for

Además del elemento recuperado, en cada iteración del array disponemos de una serie de variables que nos ofrecen valores asociados a cada uno de los elementos de la colección que estamos recorriendo.

Los valores que podemos recuperar y utilizar son los siguientes:

- ▶ **\$count**: número de elementos dentro de la colección que iteramos.
- ▶ **\$index**: índice de la iteración actual.
- ▶ **\$first**: valor boolean que nos indica si nos encontramos o no en la primera iteración.
- ▶ **\$last**: valor boolean que nos indica si nos encontramos en la última iteración o no.

Tema 9. Control Flow

- ▶ **\$even**: valor boolean que nos indica si la iteración actual es PAR.
- ▶ **\$odd**: valor boolean que nos indica si la iteración actual es IMPAR.

Podemos recuperar estos valores realizando una asignación sobre diferentes variables dentro del propio bloque FOR.

```
<ul>
  @for (estudiante of estudiantes; track estudiante.id; let cont = $count;
  let
    indice = $index; let primero = $first; let ultimo = $last; let par =
    $even;
    let impar = $odd) {
    <div>
      <p>{{ estudiante.name }}</p>
      <p>Número de elementos: {{ cont }}</p>
      <p>Índice actual: {{ indice }}</p>
      <p>¿Es la primera iteración? {{ primero }}</p>
      <p>¿Es la última iteración? {{ ultimo }}</p>
      <p>¿Es una iteración par? {{ par }}</p>
      <p>¿Es una iteración impar? {{ impar }}</p>
    </div>
  }
</ul>
```

En el ejemplo anterior hemos utilizado todas las variables para ver su funcionalidad, pero deberíamos incorporar solo aquellas que vayamos a utilizar en cada caso.

Bloque @empty

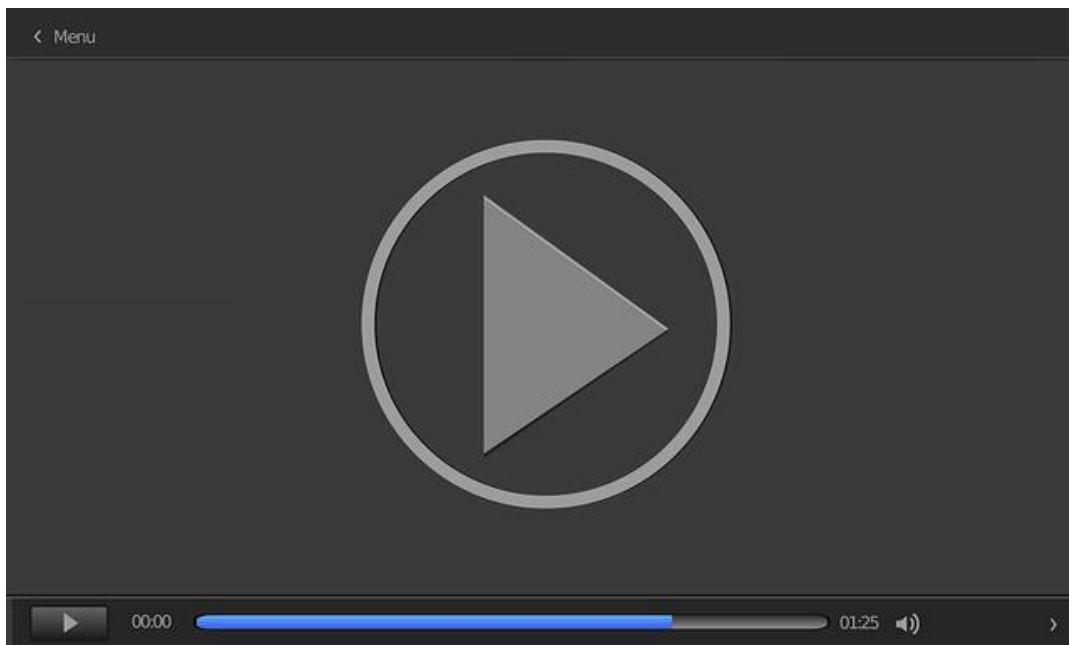
Una de las grandes mejoras asociadas al uso del bloque @for es la posibilidad de concretar un bloque de código HTML que se visualizará únicamente si el array con el que trabajamos está vacío. Esta acción la conseguimos a través del bloque @empty asociado a una iteración.

```
<ul>
  @for (estudiante of estudiantes; track estudiante.id;) {
    <li>{{ estudiante.name }}</li>
  }@empty {
    <li>No existen estudiantes</li>
  }
</ul>
```

Tema 9. Control Flow

De esta manera evitamos tener que estar lanzando condicionales previos a la iteración del bucle para comprobar el número de elementos contenidos.

En este vídeo, *Bloque de repetición @for*, veremos cómo se usa el bloque de repetición @for.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=3529af72-5c7e-4904-9ceb-b12801750652>

9.4. Bloque condicional @switch

Mediante el uso del bloque @switch podemos evaluar una propiedad y, en función del valor de esta, mostrar un bloque u otro.

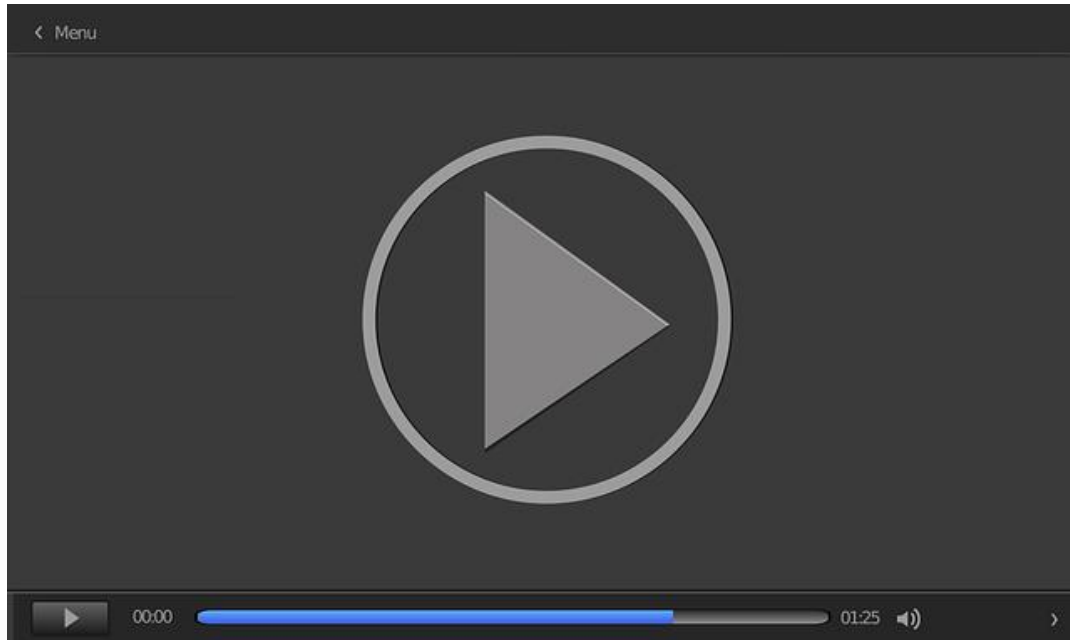
Se aproxima mucho al uso del condicional switch de Javascript. La gran diferencia con este es que no podemos evaluar condiciones complejas y nos tenemos que centrar en los valores que tengamos dentro de la clase del componente.

```
<div>
  @switch (role) {
    @case ('admin') {
      <p>Eres administrador. Todos los permisos</p>
    }
    @case ('moderador') {
      <p>Eres moderaodr. Permisos de escritura</p>
    }
    @case ('regular'){
      <p>Eres usuario. Permisos de lectura</p>
    }
    @default {
      <p>Role de usuario no admitido: {{role}}</p>
    }
  }
</div>
```

En este ejemplo estaríamos evaluando una variable llamada **role** de tipo string, definida dentro de la clase del componente. La modificación del valor de dicha variable provocaría que se viese dentro de la plantilla un párrafo u otro.

Tema 9. Control Flow

En este vídeo, *Bloque condicional @switch*, veremos cómo se usa el bloque condicional @switch.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=9415fb23-913b-448a-9684-b1280179d434>
